

17.02.2025

Eksamensrapport 2. Semester

DATA-GBG-F25A

Gruppe #6 – JavaTheHutt

PROJECT WIZARD

Projektdeltagere:

Theis Milling Eriksen - 28.06.2000 – ther0001

Andreas Bøge Holgersen - 04.02.1997 – anh00008

Niklas Holt Läu - 19.12.1997 – nila0009

Ella Costelloe Skov - 26.02.1998 – elsk0002

GitHub links:

Repository: https://github.com/TeamJabbaTheHutt/ProjectWizard

GitHub Project: https://github.com/orgs/TeamJabbaTheHutt/projects/3/views/1

Theis: https://github.com/iotuz

Andreas: https://github.com/Manden9753

Niklas: https://github.com/nikholt1

Ella: https://github.com/EllaCostello

Azure Web App Link:

http://projectwizard-ffh2amf5gqfghgez.swedencentral-01.azurewebsites.net/

Table of Contents

Indledning & Problemformulering - Theis 4

Produkt Vision - Theis 6

Systemet skal skabe værdi inden for tre overordnede områder 6

Samlet vurdering 8

Kort sagt 8

Virksomhedsanalyse – (Theis) 9

Feasibility Study 9

Definering og afgrænsning af projektet 9

Motivation / Målsætning 9

Formål med Feasibility Study 9

Samlet vurdering og anbefaling (Go/No Go) 12

Interessentanalyse 12

Udvidet Riskotabel 16

Kravspecifikation – User Stories – (Theis) 17

US#96 Sprint 1 - login for at se homepage 17

US#102 sprint 1 – Se tildelte projekter på forsiden 17

US#126 sprint 1 - Lav nyt projekt 17

US#149 sprint 2 – Registrer ny user 18

US#142 sprint 2 – Navigere til projektets dashboard og se subprojects og tasks/subtasks 18

US#153 Sprint 2 - redigere project 18

US#161 sprint 2 - lav ny task/subtask 19

US#152 sprint 2 – Projekt Statistik (Project Statistics Page) 19

Teknologivalg – (Niklas) 20

System 20

Workflows, Code Inspection og Tests 21

PaaS og Deployment 22

Teknologi Arbejdstilgang 22

UI/UX Design – (Niklas) ***24***

UI/UX Arbejdstilgang 24

Emphazise And Define 24

Ideate 24

Prototype 25

Usability test 25

Heuristic Evaluation 26

Endegyldige Design 27

Database Design & Domænemodel – (Niklas) ***30***

Domænemodel 30

ER Diagram 31

DDL 32

DML 34

Software Design – (Andreas) ***36***

Overordnet Arkitektur 36

UML Package Diagram 36

Controller-laget 36

Service-laget 36

Repository-laget 37

Model-laget 37

Exceptions 37

Class Diagram 37

Test og Kodeanalyse – (Andreas) ***41***

Kodeanalyse med Qodana 42

Scrumprocessen – (Ella) ***43***

Arbejdsprocessen faktuelt 43

Arbejdsprocessen reflekteret 48

Continuous Integration & Deployment/Delivery – (Ella) _____ *51*

Status på implementering – (Ella) _____ *53*

Acceptance Test _____ *54*

Acceptance test 1 - US#96 _____ **54**

Acceptance test 2 - US#102 _____ **54**

Acceptance test 3 - US#126 _____ **54**

Acceptance test 4 - US#149 _____ **55**

Acceptance test 5 - US#142 _____ **55**

Acceptance test 6 - US#153 _____ **55**

Acceptance test 7 - US#161 _____ **56**

Acceptance test 8 - US#152 _____ **56**

Konklusion _____ *57*

Litteraturliste _____ *58*

Bilag _____ *59*

Indledning & Problemformulering - *Theis*

Vi har I følge med vores 2. Semester eksamen fået opgaven at udvikle og designe et projektkalkulationsværktøj til firmaet Alpha Solutions.
Alpha Solutions er et dansk partnerdrevet udviklingshus, der i mere end to årtier har specialiseret sig i udviklingen af avancerede digitale løsninger inden for e-commerce, CRM, datahåndtering og marketing automation. Med mere end 80 konsulenter fordelt på flere lande leverer virksomheden skræddersyede it-løsninger til store nationale og internationale virksomheder, hvor kompleksiteten ofte er høj, og hvor kravene til planlægning, kvalitet og struktur er afgørende.
I en organisation af denne størrelse, hvor projekter, teams og teknologier konstant udvikler sig, bliver effektiv projektstyring en central faktor for succes. Når mange udviklere arbejder parallelt på forskellige leverancer, opstår der hurtigt udfordringer med at bevare et klart overblik over status på opgaver, hvem der arbejder på hvad, hvor langt man er i processen, og om tids estimaterne er realistiske.

Hvordan kan vi udvikle et projektkalkulationsværktøj, der gør Alpha Solutions i stand til effektivt at nedbryde projekter, estimere tidsforbrug og skabe overblik over opgaver, deadlines og ressourcer – samtidig med at systemet understøtter virksomhedens projektorienterede arbejdsform og kan anvendes af udviklere i deres daglige planlægning?
For at besvare denne problemformulering undersøger projektet:

- Hvilke krav der skal opfyldes for at understøtte Alpha Solutions' behov for projektnedbrydning, opgavehåndtering og tidsestimering

- Hvordan user stories, domænemodel og database kan konstrueres, så de afspejler problemdomænet

- Hvordan UX/UI-design kan sikre, at værktøjet er brugervenligt og understøtter de arbejdsgange, udviklerne faktisk har

- Hvordan systemet kan udvikles iterativt med Scrum og dokumenteres gennem sporbarhed mellem krav, design og kode

- Hvordan løsningen kan testes, deployes og gøres teknisk bæredygtig for fremtidig videreudvikling

Produkt Vision - *Theis*

CTO Klaus Petersen beskriver i virksomhedens præsentation, hvordan hans rolle kræver et tydeligt indblik i:

- hvad virksomheden **allerede ved** – deres kompetencer, erfaring og interne kapacitet
- hvad virksomheden **mangler at vide** – om der er huller i planlægning, estimater og forventninger
- hvordan dette **matcher kundernes behov** – så der skabes reel værdi for slutkunden

For at understøtte denne tilgang søger virksomheden et system, der kan bidrage til en mere struktureret og datadrevet måde at arbejde projektorienteret på. Deres håb er, at et internt projektstyringsværktøj kan fungere som en central platform, hvor viden samles, og hvor både udviklere og ledelse kan få et retvisende og opdateret overblik over projekternes fremdrift.

Systemet skal skabe værdi inden for tre overordnede områder

1. **Forbedret projekt gennemsigtighed**

   En af de største udfordringer i større udviklingsorganisationer er manglen på et samlet billede af projekternes status. Ved at indføre et internt projektværktøj bliver det muligt for Alpha Solutions at visualisere:

   - hvilke projekter der er eksistere
   - hvilke opgaver de består af
   - hvem der er ansvarlige
   - hvor langt hver opgave er i processen
   - hvor der er risiko for forsinkelser eller flaskehalse

   Denne gennemsigtighed styrker ikke blot den daglige koordinering, men giver også ledelsen mulighed for at træffe bedre og hurtigere beslutninger baseret på reelle data frem for antagelser.

Gennemsigtighed er essentielt i en virksomhed med mange parallelle projekter, da det reducerer behovet for manuelle opdateringer, unødvendige møder og afhængigheder mellem teams.

2. **Bedre estimering og planlægning**
   Estimering er en kerneaktivitet i Alpha Solutions' arbejde. Som det fremgår af kickoff-præsentationen, er nedbrydning, vurdering og opfølgning fundamentet i deres projektorienterede tilgang. Værktøjet understøtter denne proces ved at:
   - opdele projekter i **subprojects**, **tasks** og **subtasks**
   - give mulighed for at registrere både **estimeret** og **faktisk tidsforbrug**
   - give overblik over **ressourcer** og **ansvarlige udviklere**
   - skabe struktur for **løbende opdateringer** og status

   Denne systematiske opdeling hjælper ikke alene teams med at forstå opgaven bedre, men gør estimeringer mere præcise over tid. Det betyder også, at erfaringer fra tidligere projekter lettere kan genbruges, så fremtidige estimater kan blive endnu mere realistiske.
   For en CTO, der arbejder med at forene virksomhedens kompetencer med kundernes behov, giver et sådant databaseret værktøj et stærkt fundament for strategiske beslutninger.

3. **Effektiv kommunikation og samarbejde**
   Et centralt mål for systemet er at reducere kommunikationsfejl og forbedre samarbejdet mellem udviklere, projektledere og ledelsen. Når information er samlet ét sted, undgår virksomheden:
   - **Tab af kontekst** når projekter overdrages
   - **Manuelle koordinationsprocesser**, der tager tid og skaber risiko for fejl

   Systemet gør det let at se:
   - hvem der arbejder på hvad
   - hvordan en opgave relaterer sig til den overordnede løsning
   - om ressourcer skal flyttes
   - om deadlines kan overholdes

   Det skaber en fælles forståelse af projektets status og gør det lettere at opdage og løse problemer tidligt.

Samlet vurdering

Systemet fungerer som et centralt projektværktøj, der skal:

- styrke virksomhedens interne struktur
- give bedre overblik over komplekse projekter
- forbedre estimeringsprocesser
- øge samarbejde og videndeling
- sikre mere effektive arbejdsgange på tværs af udviklere, teams og ledelse

Kort sagt

Systemet er designet til at hjælpe Alpha Solutions med at planlægge, nedbryde, estimere og gennemføre komplekse IT-projekter mere effektivt — og dermed levere højere kvalitet og bedre kundeværdi.

Virksomhedsanalyse – *(Theis)*

Feasibility Study

Definering og afgrænsning af projektet

Projektet går ud på at udvikle et simpelt projektstyringssystem til Alpha Solutions, inspireret af GitHub Projects .
Formålet er at skabe et internt værktøj, der gør det lettere for medarbejderne at planlægge, fordele og følge op på opgaver uden unødvendig kompleksitet.

Motivation / Målsætning

Alpha Solutions oplever, at eksisterende projektstyringsværktøjer som GitHub Projects, smartsheet.com og projectmanager.com kan være for tunge og uoverskuelige til mindre projekter.
Målet er at udvikle en lettere og mere skræddersyet løsning, der passer til virksomhedens behov og arbejdsgange.

Formål med Feasibility Study

Formålet er at vurdere, om projektet er realistisk at gennemføre ud fra tekniske, økonomiske, organisatoriske og juridiske faktorer, samt om det kan leveres indenfor den fastsatte tidsramme.

Teknisk feasible

Systemet udvikles som en webapplikation (springboot, thymeleaf) med HTML, CSS og Java.[Teknologivalg – *(Niklas)*]
Alpha Solutions' eksisterende infrastruktur understøtter disse teknologier, og gruppen har kompetencerne til at implementere løsningen.
Der skal sikres grundlæggende brugerlogin. Teknologien vurderes stabil og realistisk.
Projektet er teknisk feasible.

Økonomisk og finansiel feasible

Projektet er et studieprojekt og kræver derfor ingen reelle økonomiske investeringer fra Alpha Solutions.

Hvis systemet senere skal implementeres, kan det gøres billigt via eksisterende serverløsninger eller cloud-hosting.
Fordelen for virksomheden vil være et mere målrettet værktøj med lavere vedligeholdelsesomkostninger end større standardløsninger.
Såfremt virksomheden vælger at fortsætte med at bruge Azure som PaaS for at hoste systemet. Ud fra azure pricing[1] og calculator[2] og vores forventninger til hvor stort projektet og anvendelsen potentielt kunne være, kan vi se at systemet kan komme til at koste op til 54 dollars om måneden, sammenlagt med en større plan og databasehåndtering kan det ende op på omkring 80 dollars om måneden.
Vi vil dog råde virksomheden til at anvende gratis plan af azure til at teste programmet og derudfra konstatere om det er noget de gerne vil implementere, for efterfølgende at anvende On-Site hosting til at tilgå systemet fremover, dette vil koste dem mindre, og så har de også fuld rådighed over systemet på deres egne servere.
Projektet er økonomisk feasible.

Operationel og organisatorisk feasible

Løsningen vil understøtte Alpha Solutions' nuværende arbejdsprocesser og skabe bedre overblik for medarbejderne.
Systemet designes med fokus på brugervenlighed, så overgangen fra nuværende arbejdsmetoder kræver minimal oplæring.
Der forventes god accept, da programmet tilpasses deres konkrete behov.
Projektet er organisatorisk feasible.

Retlig og juridisk feasible

Systemet vil blive udviklet i overensstemmelse med gældende GDPR-regler for håndtering af brugerdata.
Der anvendes udelukkende open-source værktøjer og frameworks med frie licenser, hvilket eliminerer risikoen for licensbrud eller ophavsretlige problemer.

Som vi kan se ud fra vores licenses, er de primært open source og frit tilgængelige. Ved eventuel kommerciel anvendelse er det væsentligt at sikre overholdelse af de respektive licenser.

Den licens, vi skal være særlig opmærksomme på, er (*GPL v2*) Teknologivalg – *(Niklas),* som er strong copyleft-licens. Det vil sige at at hvis software distribueres og linker til (*GPL v2*), skal den distribuerede kode som udgangspunkt også være under samme licens.

[1] https://azure.microsoft.com/en-us/pricing/details/app-service/static/
[2] https://adsense.google.com/start/ - calculator

Systemet er som udgangspunkt udviklet under Alpha Solutions, hvilket betyder at vi ikke distribuerer programmet fordi Alpha Solutions per definition har udviklet programmet. Derved kommer vi ikke i konflikt med (*GPL v2*) hvilket betyder at firmaet eller vi selv kan bestemme hvilken licens vores system skal have.

Webapplikationen er hosted på (Azure PaaS) hvilket vil sige at hvis Alpha Solutions vil bringe systemet i brug, skal de overføre det fra vores "Student Program" til et "coorporate Azure-konto", for at overholde Azures cloud-tjeneste regler.

Realistisk set, ud fra at det er et eksamens projekt, så er vores program fuldt ud open source og bliver ikke distribueret, derfor overholder vi alle vores licenser.
Det eneste vi skal være opmærksomme på, er at vi ud fra vores licenser skal angive at vi har brugt de licenses såsom Apache 2.0 der kræver tilføjelse af denne information i projektets *LICENSE* eller *NOTICE*-fil, baseret på Apache's licensskabelon.
Siden projektet anvender (*maven*) til afhængigheds styring, kan notitsen genereres automatisk, vi skal dog sikre at notitsen er til stede og overholder licenskravene.

I og med at vi bruger maven til afhængigheds regulering, kan vi integrere at den automatisk laver notitsen, vi skal dog sikre os at den er til stede og er tilstrækkelig.
Projektet er juridisk feasible.

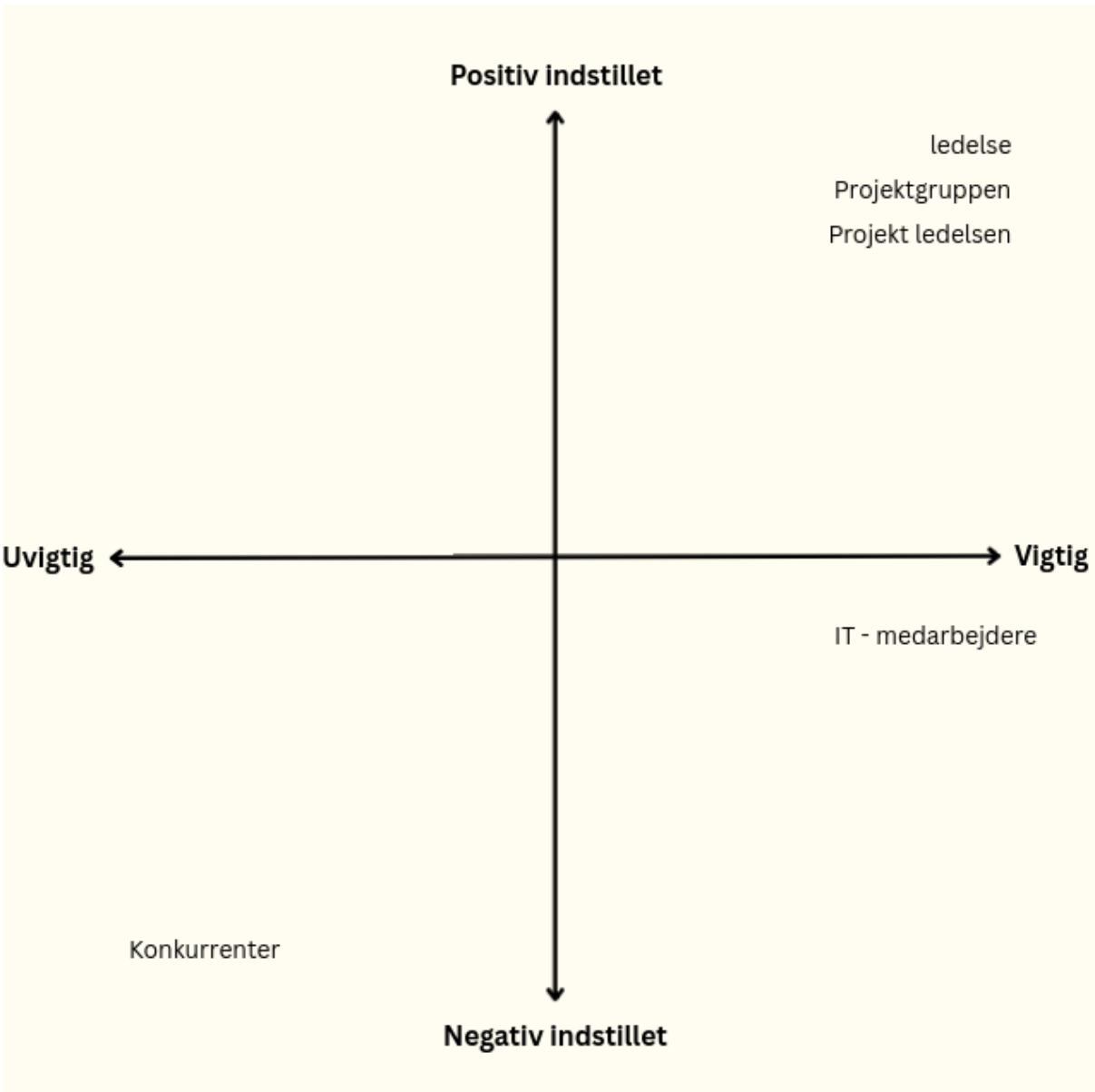
Vurdering i forhold til andre løsningsmuligheder

Eksisterende alternativer som GitHub Projects, smartsheet.com og projectmanager.com tilbyder mange funktioner, men kan virke komplekse for Alpha Solutions' brug.
Vores løsning adskiller sig ved at være målrettet virksomhedens specifikke behov, og mindre avanceret.
Projektet udføres internt som en del af uddannelsesforløbet, hvilket giver fleksibilitet og læring uden ekstra omkostninger.
Såfremt virksomheden vurdere at systemet giver dem en fordel frem for at anvende gratis/billige og større, dog mere komplekse alternativer, vurderer vi at projektet er feasible i forhold til andre løsningsmuligheder.

Samlet vurdering og anbefaling (Go/No Go)

Projektet vurderes som **gennemførligt** både teknisk, økonomisk og organisatorisk.
Alpha Solutions kan drage fordel af et simpelt og skræddersyet værktøj, som kan forbedre intern opgavestyring.
Derfor anbefales en **Go-beslutning** for projektet.

Interessentanalyse



Interessent	Fordele ved projekt	Ulemper ved projekt	Vurdering af interessant	Håndtering
IT-medarbejderne	Konkurrenterne kan indirekte fungere som inspiration, da deres eksisterende løsninger viser, hvilke funktioner brugerne efterspørger, og hvad der fungerer godt i markedet.	Hvis systemet er dårligt dokumenteret eller teknisk ustabilt, kan det skabe ekstra arbejde. En ny platform kan også kræve oplæring eller ændrede arbejdsgange. Såfremt at de har behov for flere features end programmet har implementeret, kræver det at videreudvikle programmet, så det rammer de nye behov som er stillet.	IT-medarbejderne er en **vigtig interessent**, da de står for drift, support og fremtidig vedligeholdelse. Deres accept er nødvendig for et stabilt og langsigtet system.	Involver dem tidligt i design og tekniske beslutninger. Vi skal sørge for tydelig dokumentation, gode testprocedurer og mulighed for at give feedback løbende.
Konkurrenterne	Konkurrenterne kan indirekte fungere som inspiration, da deres eksisterende	De er en udfordring, fordi de allerede har stærke,	Konkurrenterne er en **ekstern interessent med næsten ingen påvirkning**, da de	Analyser deres svagheder og differentierer os ved at skabe en mere simpel og

	løsninger viser, hvilke funktioner brugerne efterspørger, og hvad der fungerer godt i markedet.	kendte og etablerede systemer. De kan hurtigt tilpasse sig og efterligne nye funktioner, hvilket kan gøre det svært for et nyt produkt at skille sig ud.	ikke deltager i projektet, men har stor betydning for markedet og brugerforventninger.	brugervenlig løsning. Vi skal fokusere på brugervenlighed og UX ud fra de udfordringer projektledelsen fortæller os de har. Systemet skal være mindre komplekst og skal fokusere på at automatisere de insights som konkurrenterne ikke har.
Projektledelsen	Projektledelsen kan begynde med en mere struktureret dagligdag, da de får et mere overskueligt program som er specifikt udviklet til deres behov.	De ender med ikke at have ligeså mange features som de plejede at have. Dette kan ende med utilfredshed hos projektledelsen.	Projektledelsen er en **intern nøgleinteressent**, da de skal anvende programmet når slut produktet er færdigt. De skal bruge det til at planlægge og vedligeholde deres arbejdsopgaver.	Løbende vise demoer eller hvordan produktet er blevet videreudviklet, da det er dem der skal bruge programmet i sidste ende. Være opsøgende omkring de features som de er mest interesseret i.
Ledelsen	Projektet kan øge effektiviteten, skabe bedre overblik og modernisere arbejdsprocesserne. Et succesfuldt system styrker	Hvis projektet fejler, kan det medføre ressourcespild, utilfredse medarbejdere og et ineffektivt	Ledelsen er prioriteret **og strategisk interessent**, da de finansierer projektet og sætter retningen. Deres tilfredshed er	Sørg for klare statusmøder, forventningsafstemning og dokumentation af fremdrift. Kommunikér risici tidligt og fremlæg løsninger, så

	virksomhedens interne struktur og konkurrenceevne.	arbejdsflow. Der er også risiko for forsinkelser og ekstra omkostninger.	afgørende for projektets fortsatte opbakning.	ledelsen bevarer tilliden til projektgruppen.

Interessent (hvem)	Hvad	Hvornår	Hvordan	Ansvarlig
IT - Medarbejderne	Drift og vedligeholdelse	Under udvikling og ved overdragelse	Dokumentation og demonstrationer	Projektgruppen / projektleder
Konkurrenterne	Krav og forventninger til projektet	Under analyse og projekt design	Markedsresearch og funktionssammenligning	Projektleder
Projektledelsen	Sikre fremdrift, kvalitet og koordinering.	Hele projektet.	Statusmøder og milepælesopfølgning.	Projektleder
Ledelsen	Ønsker af effektivt og modernisered e system.	Ved start, større beslutninger og godkendelser.	Kort rapportering og præsentationer.	Projektleder.

Udvidet Riskotabel

Risikoanalyse tabel							
Risikomoment	Sandsynlighed	Konsekvens	Produkt	Præventive tiltage	Ansvarlig	Løsningsforslag	Ansvarlig
Nøgleprojektdeltagere forlader projektet	2	7	14	Sørge for at vi har en åben og ærlig dialog, og et generelt godt arbejdsmiljø.	Projektledere & projekt-deltagerne.	Hvis en projektdeltager forlader projekt har vi sørget for at denne ikke alene har bestemt viden. Alle dokumentere deres arbejde og en aftale er på plads for, hvordan vi håndtere overdragelse af opgaver.	Projektledere & projekt-deltagerne.
Ændringer i kravene til system mens det udvikles	2	10	20	At lave møder med kunden og sikre at der er en klar forventningsafstemt plan.	Projektgruppen og Kunden	Der laves en kontrakt der tager højde for ændringer, og at fakturering er baseret på det initielle produkt der er aftalt.	Projektlederen og kunden.
Mangelfulde testrunder	2	3	6	Sørge for at alle dele testes løbende. At vi ikke sætter noget i produktion der ikke er godkendt.	Projektleder og projekt-deltagerne	Vi har efter aftale mulighed for at trække på en 'tester' som kan informere os når fejl dukker op som følge af mangelfuld test.	Projektlederen og chef for testperson
Interne konflikter/uenigheder	3	3	9	Have en klar aftale om hvordan uenigheder/konflikter skal løses.	Projektleder og projekt-deltagere	Indsæt møde med en upartisk 3. part der neutralt kan hjælpe med at føre ordentlig dialog og finde en løsning.	Projektleder og upartisk 3. part
Problemer hos eksterne værktøjer (Azure, Github etc.)	2	7	14	Grundig undersøgelse af udbydere, med en forsikring der dækker outages mv.	Projektleder	Baseret på varighed og omfang, før dialog med udbyderen og i værste fald hav en plan B for alternativer (AWS f.eks)	Projektleder og udbyder

Kravspecifikation – User Stories – *(Theis)*

US#96 Sprint 1 - login for at se homepage

Som bruger vil jeg kunne se homepage når du er logget ind.

Acceptkriterier

- Brugeren kan indtaste email og adgangskode
- Systemet validerer login-oplysninger
- Ved korrekt login viderestilles brugeren til Home page
- Ved forkert login vises en fejlmeddelelse

US#102 sprint 1 – Se tildelte projekter på forsiden

Som bruger vil jeg kunne se alle projekter, jeg er medlem af, når jeg logger ind, så jeg får et hurtigt overblik over mit arbejde.

Acceptkriterier

- Brugeren vises en projektoversigt efter login.
- Systemet viser kun projekter, som brugeren er tilknyttet.
- Hvis brugeren ikke er medlem af nogen projekter, vises en passende besked.

US#126 sprint 1 - Lav nyt projekt

Som bruger vil jeg kunne oprette et nyt projekt, så jeg kan planlægge og strukturere arbejdet.
Acceptkriterier

- Brugeren kan tilgå en "Create New Project"-formular
- Formularen indeholder:
    - Projekttitel
    - Beskrivelse
    - Deadline
- Systemet validerer input ved oprettelse
- Projektet gemmes i databasen ved gyldig input
- Brugeren viderestilles til homepage hvor projekterne kan ses

US#149 sprint 2 – Registrer ny user

Som ny bruger vil jeg kunne oprette en brugerprofil, så jeg kan få adgang til systemet og blive tilknyttet projekter.

Acceptkriterier

- Brugeren kan indtaste navn, email og adgangskode
- Systemet validerer, at email ikke allerede eksisterer
- Ved gyldig input oprettes brugeren i systemet
- Brugeren kan efterfølgende logge ind
- Ved fejl vises en passende fejlmeddelelse

US#142 sprint 2 – Navigere til projektets dashboard og se subprojects og tasks/subtasks

Som bruger vil jeg kunne klikke på et projekt i min projektliste og blive taget til projektets dashboard og se de subprojekts og tasks/subtasks

Acceptkriterier

- Trykker på projektet og bliver navigeret til dette projekts dashboard
- Alle brugerens subprojects og tasks/subtasks vises som klikbare elementer der udfolder sig, så mere information bliver givet.

US#153 Sprint 2 - redigere project

Som projektmedlem vil jeg kunne redigere et projekt, så jeg kan holde projektet opdateret.

Acceptkriterier

- Brugeren kan redigere projekttitel, beskrivelse og deadline
- Brugeren kan vælge at ændre medlemmere
- Brugeren kan indtaste email på et medlem der ønskes fjernet
- Systemet validerer om brugeren eksisterer
- Medlemmet fjernes fra projektet ved succes
- Ved ugyldig email vises en fejlmeddelelse

US#161 sprint 2 - lav ny task/subtask

Som projektmedlem vil jeg kunne oprette nye opgaver i et projekt, så jeg kan nedbryde arbejdet i mindre dele.

Acceptkriterier

- Brugeren kan angive:
  - Titel
  - Beskrivelse
- Som gemmes i databasen og bliver vist under project

US#152 sprint 2 – Projekt Statistik (Project Statistics Page)

Som bruger vil jeg kunne se statistiske overblik over projektets fremdrift, såsom brugte timer, resterende estimerede timer, antal færdige tasks, procent-færdig og total task-overblik.

Acceptkriterier

- Statistikside viser totale estimerede timer, faktiske timer, resterende timer.
- Viser antal opgaver færdige vs. total.
- Viser % fuldført baseret på tasks.
- Statistikker opdateres dynamisk efter databasens data.

Teknologivalg – *(Niklas)*

Vores system anvender forskellige former for teknologier som blandt andet kan findes under vores techstack på vores README.md[3] disse vil dog blive grundigere gennemgået her.

System

I systemet anvender vi:

- **IntelliJ IDEA Ultimate edition 2025.3**[4]
  (*Apache 2.0 License*)[5]
  - Development IDE

- **Java SDK jbr-21 Java Jetbrains Runtime 21.0.6**[6]
  (*GPLv2 License*)[7]
  - Overordnet programmeringssprog for backend
    - Downgraded fra Java 25 da vi oplevede DispatchServlet fejl på enkelte computere. hvor 21 fungerede på samtlige dev computere.

- **MySQL Ver 8.0.43**[8]
  (*GPL v2 License*)
  - Anvender vi til at oprette, gemme og vedligeholde data gennem DML og DDL language i incode queries. Dette bliver også beskrevet i DDL og DML sektionen.

- **Apache Maven 3.8.7**[9]
  (*Apache 2.0 License*)
  - Dependency management system til at inkludere de nødvendige pakker og bundles til vores projekt

[3] https://github.com/TeamJabbaTheHutt/ProjectWizard/blob/main/README.MD
[4] https://www.jetbrains.com/idea/
[5] https://www.apache.org/licenses/LICENSE-2.0
[6] https://www.java.com/en/
[7] https://www.gnu.org/licenses/old-licenses/gpl-2.0.en.html
[8] https://www.mysql.com/
[9] https://maven.apache.org/

- **Springboot (v3.5.7)** (Del af maven packages)[10]
  (*Apache 2.0 License*)
    - Framework til at håndtere development og production environment gennem .properties files, samt inkluderer mindre dependency bundles så som
        - Springboot-starter-web 3.5.7
        - Springboot-starter-jdbc 3.5.7
        - Springboot-starter-test 3.5.7
        - Springboot-starter-thymeleaf 3.5.7

      Til at håndtere webapplikation opsætning, in-code MySQL queries, Integration tests, weblayer slice tests og unit tests. Derudover data connector mellem backend controllers og frontend data fetch.

- **HTML5**[11]
    - Webapplikationens frontend der anvender thymeleaf til data fetch fra backend.

- **CSS3**[12]
    - Styling af webapplikationens frontend.

- **mysql-connector-j**[13]
  (*LGPL-2.1 Only License*)[14]
    - Anvendt af Springboot JDBC for at forbinde med MySQL databasen.

Workflows, Code Inspection og Tests

- **H2 Database Maven: com.h2database:h2:2.3.232**[15]
  (Mozilla Public License 2.0)[16]
    - Temporary in memory database til integration tests

[10] https://spring.io/projects/spring-boot
[11] https://html.spec.whatwg.org/multipage/
[12] https://www.w3.org/Style/CSS/
[13] https://mvnrepository.com/artifact/mysql/mysql-connector-java
[14] https://spdx.org/licenses/LGPL-2.1-only.html
[15] https://h2database.com/html/build.html?highlight=maven&search=maven - firstFound
[16] https://www.mozilla.org/en-US/MPL/2.0/

- **Qodana 2025.2 code inspection**[17]
    - Code inspection til underretning af eventuelle fejl i kode eller forhold vi skal være opmærksomme på.

- **Github Workflows**[18] **med**
- **checkout@4 version**[19] *(MIT License)*[20]
- **setup-java@4 version**[21] *(MIT License)*
    - CI-CD pipeline til execution af Qodana code inspection, samt test execution og build execution ,fra maven lifecycle fra clean til package, på push til main og eller pull requests rettet mod main.

PaaS og Deployment

- **Azure Hosting**[22]
    - Vores Github workflow deployer direkte til vores online hosted system på Azure i et student environment. Her anvendte vi Azure web app ressourcen og Azure database til at håndtere vores webapplikation.

Teknologi Arbejdstilgang

Vores development environment er opbygget gennem en switch i vores spring.active.profile i vores application.properties, der tillader os at skifte fra production til development, og angive database URL i vores environment variables.
Derved anvender vi lokale databaser til udvikling og testning af vores implementerede features.

Techstack:

IntelliJ: https://www.jetbrains.com/idea/

Java: https://www.java.com/en/

MySQL: https://www.mysql.com/

Maven: https://maven.apache.org/

[17] https://www.jetbrains.com/qodana/
[18] https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows
[19] https://github.com/actions/checkout
[20] https://opensource.org/license/mit
[21] https://github.com/actions/setup-java
[22] https://azure.microsoft.com/en-us/free/students

Springboot: https://spring.io/projects/spring-boot

HTML5: https://html.spec.whatwg.org/multipage/

CSS3: https://www.w3.org/Style/CSS/

mysql-connector-j: https://mvnrepository.com/artifact/mysql/mysql-connector-java

H2 database:
https://h2database.com/html/build.html?highlight=maven&search=maven#firstFound

Qodana: https://www.jetbrains.com/qodana/

Github workflows: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows

Github checkout@4: https://github.com/actions/checkout

Github setup-java@4: https://github.com/actions/setup-java

Azure PaaS: https://azure.microsoft.com/en-us/free/students

Licenses:

GPL v2 License: https://www.gnu.org/licenses/old-licenses/gpl-2.0.en.html

LGPL-2.1 only License: https://spdx.org/licenses/LGPL-2.1-only.html

Mozilla Public License 2.0: https://www.mozilla.org/en-US/MPL/2.0/

MIT License: https://opensource.org/license/mit

Apache 2.0 License: https://www.apache.org/licenses/LICENSE-2.0

UI/UX Design – *(Niklas)*

UI/UX Arbejdstilgang

I projektet har vi arbejdet ud fra de (10 Usability Heuristics for User Interface Design)[23] og (Shneiderman's "Eight Golden Rules of Interface Design")[24]. Her har vi gjort brug af (Interaction Design Foundation's The Gestalt Principles)[25], til at opbygge et wireframe, hvor vi primært har arbejdet med (de 4 gestalt principper)[26].

Emphazise And Define

Emphasize og define faserne er defineret ud fra problemstillingen som Alpha Solution gav os, hvilket gav os muligheden for hurtigt at gå videre til ideate fasen.

Ideate

I ideate fasen valgte vi at arbejde ud fra en "collaborative" metode, hvor hele gruppen sad sammen og i fællesskab startede med at lave skitser i papirform, som kan findes under Bilag 1 over hvordan systemet skulle fungere og se ud. Derefter afprøvede vi vores ideer på ét fælles wireframe, hvor til vi anvende (*Canva*)[27]. (*Canva*) tillader at flere brugere samtidigt kan arbejde på det samme wireframe.
Efterfølgende gennemgik og vurderede vi de enkelte "pages" og deres elementer ud fra (*de 4 gestalt principper*).
På baggrund af denne proces valgte vi et minimalistisk design med stort fokus på designprincipperne, proximity og similarity.
Det udarbejdede wireframe kan findes under Bilag 5 .

[23] https://www.designprinciplesftw.com/collections/10-usability-heuristics-for-user-interface-design
[24] https://www.designprinciplesftw.com/collections/shneidermans-eight-golden-rules-of-interface-design
[25] https://www.interaction-design.org/literature/topics/gestalt-principles?srsltid=AfmBOop-QOCBP9FbYwBv95KvMjXcg7GCD--ZioluFGcpm1-R0-ZkEIm8
[26] https://github.com/EK-DATA-2SEM-PROGSYSTEK/DATA-GBG-F25A-B/blob/master/Uge43/2.gang/gestaltlove.jpg
[27] https://www.canva.com/

Prototype

Prototypen, der blev udviklet ud fra vores wireframe inden første usability test, kan findes i bilag [Bilag 11]

Usability test

Efter implementeringen af vores design i programmet udførte vi en usability test hvor vi anvendte (*NN/g's Usability (User) Testing 101*)[28] til at udarbejde 6 tasks som vores "participants" skulle udføre i det funktionelle program.

Usability Testen blev udført ved at vi som "Facilitators" udarbejdede 6 tasks bestående af:

1. Registrer en ny bruger.
2. Opret et nyt projekt med en deadline om 2 dage.
3. Opret en subtask til en task.
4. Tilføj en assignee til en subtask og sæt det til Done.
5. Sæt en time-estimate på 2 timer og actual til 1 time på en task.
6. Se statistik og hvor mange timer vi har sparet på statistik, slet en task og se statistik igen.

Herefter lod vi vores 4 deltagere udføre de tasks i rækkefølge ud fra "think out loud" metoden. Deltagerne snakkede højt om hvad de tænkte og hvad de gerne ville trykke på for at navigere rundt i vores program.
Såfremt de var i tvivl om hvad de skulle gøre for at udføre opgaven, lod vi dem finde frem til løsningen selv.
Herefter spurgte vi dem ind til hvilke områder de havde sværest ved, og om de havde nogle løsningsforslag for at gøre det mere intuitivt.

Efter undersøgelsen udarbejdede vi et ark med

- Kategorien: Hvor i programmet problemet lå

[28] https://github.com/EK-DATA-2SEM-PROGSYSTEK/DATA-GBG-F25A-B/blob/master/Uge 43/2.gang/Design-thinking-101-NNG.pdf

- Problemstilling: hvad problemet var
- Optimeringstiltag: Deres forslag og eller vores egne anbefalinger til at løse design problemet
- Effekt: Vores vurdering af hvor stor indflydelse problemet har på vores UX.
- Omkostning: Hvor lang tid det vil tage os at implementere justeringen
- Prioritering: Sammenlagt af Effekt og Omkostning.

Arket for Usability testen findes i Bilag 4

Efter Usability testen implementerede vi:

- Dashboard: En pil der indikerer muligheden for drop down.
- Statistic: En knap til at returnere tilbage til det projekt man ser statistikkerne for.
- Dato vælger: Implementer navn over input feltet og eventuelt en besked
- Time estimate og actual: implementer fix der sørger for at hvis feltet er tomt, bliver det automatisk sat til 0.0
- Landing Page: "Go To Login" knap på landingpage

Derved opnåede vi en cirkulær implementeringsmetode ud fra (*NN/g's Design Thinking 101*). Ved først at overføre vores empathize og define til ideate hvor vi ud fra vores wireframe producerer vores prototype. Denne bliver testet af vores Usability test, hvorefter vi implementerer justeringerne. Prototypen efter justeringerne kan findes her Bilag 12

Heuristic Evaluation

Efter usability testen evaluerede vi vores design med de nye implementeringer ved hjælp af (*NN/g's How to Conduct a Heuristic Evaluation*)[29] og (*NN/g's Heuristic Evaluation Workbook*)[30] til yderligere at raffinere vores program design og implementere de sidste justeringer for at få et bedre UI / UX.

Klargøring

Først klargjorde vi vores projekt til at blive evalueret ved at inputte test data, så vi havde elementer at arbejde ud fra. Vi forberedte en tabel som vi udarbejdede ud fra (*NN/g's*

[29] https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/
[30] https://media.nngroup.com/media/articles/attachments/Heuristic_Evaluation_Workbook_1_Fillable.pdf

Heuristic Evaluation Workbook) hvor vi ligeledes implementerede (*Shneiderman's "Eight Golden Rules of Interface Design"*)ind i tabellen, for at have plads til flere issues og recommendations. Vores workbook kan findes under bilag [Bilag 14]

Scope

Vi satte vores scope til at være én heuristic ad gangen, hvor vi gennemgik programmet og ledte efter issues, derved kunne vi indsnævre vores scope og analysere designet mere fokuseret.

Samle Identificerede problemer

Når vi havde gennemført gennemgangen, skrev vi problemerne ind i vores tabel og diskuterede metoder til at rette op på designet for at løse problemstillingen.

Implementering

Efter evalueringen vurderede vi hvilke implementeringer vi kunne nå at lave inden deadline for projektet. Herunder valgte vi:

- "3. Ændre farven på statistics til grøn og edit til blå, lad delete forblive rød så den tydelig indikerer en definitiv handling."

- "4. Implementer tilbage knap på siderne, der tager brugeren ét step tilbage i deres proces."

- "11. Uddyb eventuelle processer, såsom hvordan tilføjelsen af andre brugere til et projekt påvirker deres homepage."

Her afsluttede vi 2. omgang af den cirkulære implementeringsmetode ud fra (*NN/g's Design Thinking 101*), hvor vi gennemgik "understand" og "materialize" faserne to gange.
Vi havde stadig nogle issues vi gerne ville have implementeret ud fra heuristic evalueringen. Dog gjorde vores deadline at vi måtte prioritere hvilke issues vi havde mulighed for at implementere for at fuldføre den sidste implementeringsfase.

Endegyldige Design

Efter implementeringerne af de to tests, kom vi frem til et endeligt design der kan findes i bilag [Bilag 13]

Homepage
På vores homepage, hvor brugeren kan se sine egne projekter og projekter som brugeren er del af, kan vi se at vi har lavet en minimalistisk og mørk navbar, som ikke tager fokus fra sidens egentlige funktionalitet.
Vi har placeret projekterne horisontalt som en collective i midten af vinduet, hvor de enkelte projekter hver især bliver vist som blå kasser.
Vi har lavet en kasse med samme form og størrelse til "add new project" knappen. Vi har her brugt "similarity" af kassens form og størrelse til at vise at den hører til projekterne, imens at dens anderledes farve indikerer en anden funktion end resten af kasserne.

Derved anvender vi "proximity" af de projektorienterede features som alle er placeret i midten af skærmen med en box collection layout.

Statistics
Her har vi igen stor fokus på similarity og proximity.
Alle de overordnede statistikker for hele projektet bliver vist som større bokse i venstre side af skærmen, her har de alle den samme form og farve, med forskellig data inden i.
Vi bryder konjunkturen ved at lave et tomt stykke med overskrifter for start af projektet samt deadlinen. Derved indikerer vi til brugeren at nye informationer følger.
Under deadlinen har vi placeret to loading bars for dage tilbage til deadline og hvor mange tasks der er i done ud af alle tasks i projektet. Loading barerne bryder den genkendelige farve med lyserød farve, som vi har valgt ud fra Alpha Solutions to farver.
Under loading barerne finder vi flere bokse for sub projekterne, dog indikerer deres nye farve og kassens rektangulære form, frem for firkantede form, at informationen de indeholder har betydning for noget andet end de firkantede og mørkere bokse.
Derved opnår vi forskellige grupperinger af informationer, ved at benytte proximity og similarity i forbindelse med forskellige dele af projektet.

Dashboard
Vi anvender proximity til at gruppere de forskellige elementer, der hører til ét projekt. En lys hvid kasse bruges til at indikere, at alt information inden for denne ramme tilhører det subprojekt, som er foldet ud.
Tasks er yderligere adskilt ved hjælp af en lyseblå baggrund, mens tilhørende subtasks vises direkte under den relevante task i en tabel. Denne opdeling gør det tydeligt for brugeren, hvilke elementer der hører sammen.

Similarity anvendes ved at knapperne til add subproject, add task og add subtask har den samme grønne farve, der vil signalere over for brugeren, at der er tale om tilføjende handlinger.
Statistics knappen er ligeledes grøn, da den indikerer, at brugeren kan tilgå yderligere information om projektet.

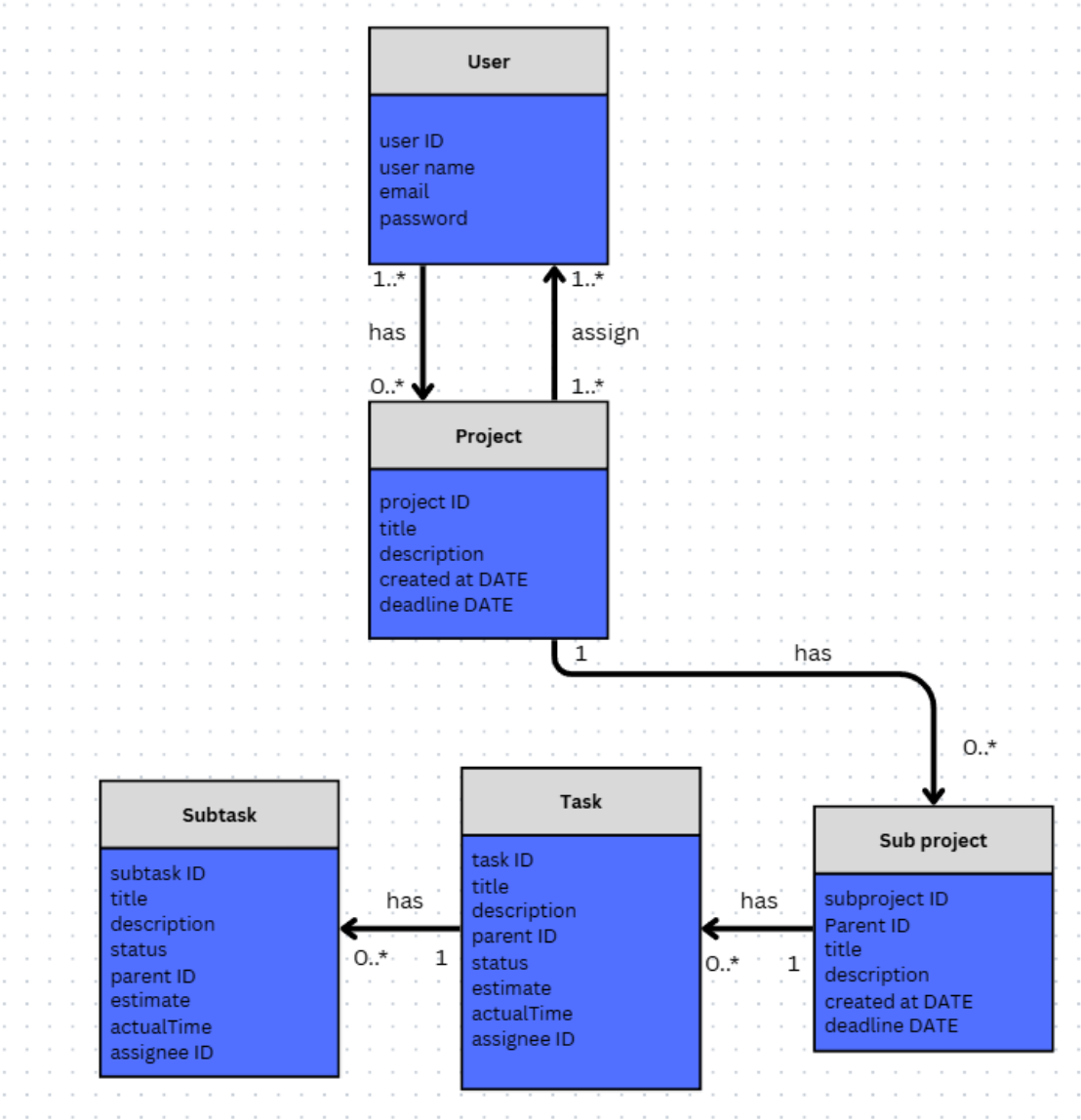
Edit-knapperne er konsekvent markeret med blå farve på tværs af siden, hvilket tydeliggør at de har samme funktion.
Delete knappen er markeret med en rød farve for tydeligt at signalere, at der udføres en endelig og potentiel destruktiv handling.
I forhold til continuity er siden opbygget med et vertikalt flow, hvor informationen præsenteres fra top til bund. Projektets hierarki understøttes ved, at subprojekter vises øverst, når den er foldet ud, efterfulgt af task og deres tilhørende subtasks. For hver lag man udfolder, rykker linjen sig en smule til højre, for at indikere en gradvis hierarkisk neddykning i elementerne.

Edit sections
Proximity anvendes ved at relaterede input felter er placeret tæt på hinanden. Titel, beskrivelse, status, estimeret og faktisk tidsforbrug er samlet i én sammenhængende sektion. Dette signalerer tydeligt at felter tilsammen udgør en samlet redigering af en fuldendt task, og såfremt tasken ikke er færdig, så kan den retmæssigt forblive uhel.
Yderligere input såsom assignee er opdelt, da den ikke udgør en præcis information om selve task eller subtask.
Similarity anvendes ved at samme design gen går ved alle former for edit, både ved Projekt, Subprojekt, Task og Subtask er samme design anvendt med forskelligt relevant information, afhængig af hvilken del man gerne vil justere. Dette giver en familiær følelse og gør at brugeren hurtigere vil vende sig til og forstå programmet, da de kun skal forholde sig til én design form.
I forhold til continuity er siden opbygget med et vertikalt flow fra top til bund, som guider brugerens blik naturligt gennem redigeringsprocessen.

Database Design & Domænemodel – *(Niklas)*

Domænemodel

Domænemodellen viser grundlaget for vores system og de relationer systemet har. Vores model tager udgangspunkt i, at alle brugere kan arbejde på alle projekter i systemet. Projekterne er opdelt i subprojekter, tasks og subtasks.
Dette skaber en fleksibilitet for brugerne af systemet, og et hierarkisk forgrenings princip, ved at hvert element der tilhører et projekt, kan findes gennem projektets id.
Modellen danner grundlaget for resten af systemet, uden at specificere implementeringen.



Som der kan ses på Bilag 2 så har vores model udviklet sig igennem projektet. Vi har gennem hele forløbet haft samme user til projekt forhold, dog har forholdet til hvad der udgjorde en task ændret sig.
Vi implementerede subprojekt og planlagde at lade subprojekt have samme funktion som en task, hvor alle elementer havde projekt_id som de kunne blive fundet igennem.
Vi evaluerede vores model og kom frem til at det ikke var den optimale måde at finde vores data på, og at projekt_id i alle elementer eventuelt var redundant.
Derfor kom vi frem til den sidste model, hvor vi besluttede os for, at projektet skal have et subprojekt, før det kan få tasks med subtasks. Derved kunne vi fjerne projekt_id på alle elementerne og find frem til subtasks gennem parent_id.
Derudover tilføjede vi created_at datoer til sidst, da det gik op for os at vi manglede noget data til statistikkerne.

ER Diagram

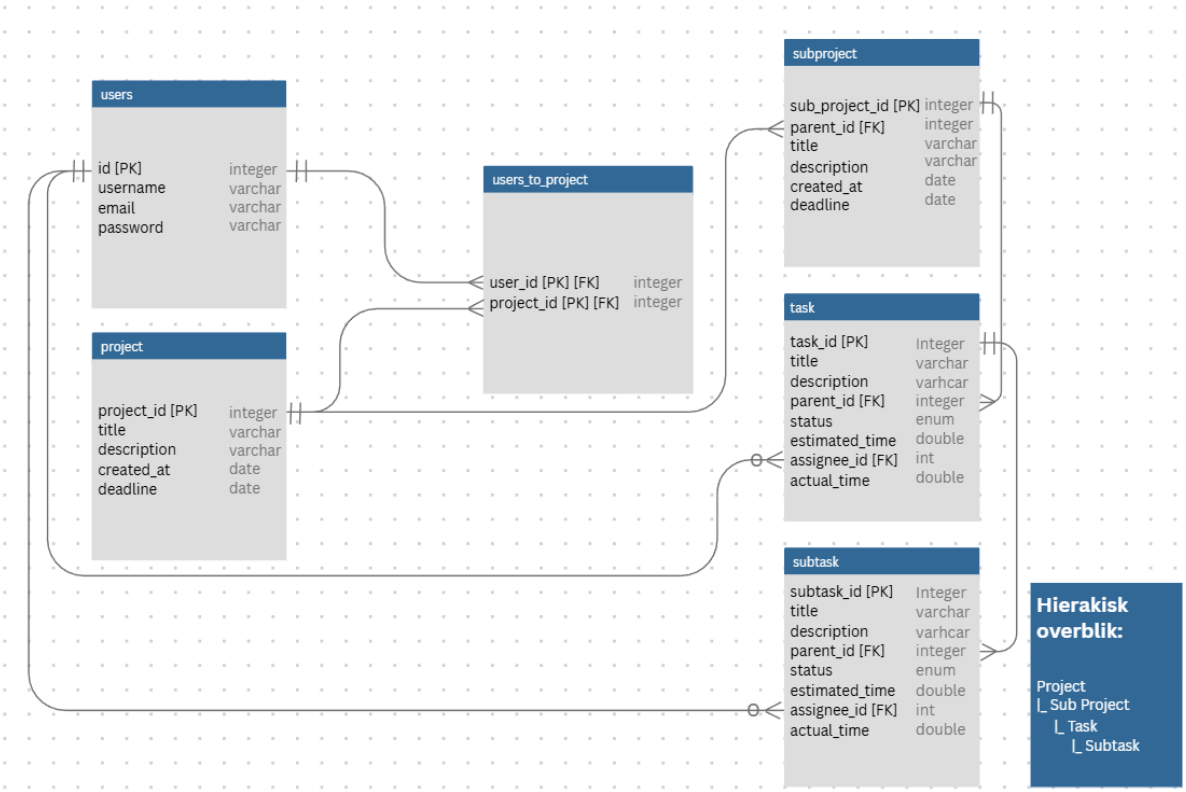
Vores Entity Relationship diagram viser ligesom domain modellen et forhold mellem de forskellige tabeller. ER-diagrammet fungerer som et blueprint til hvordan vores database skal opbygges og hvilke forhold de forskellige primære nøgler og fremmednøgler udgør, samt hvordan vi håndterer vores tabellers forhold til hinanden.
Brugere er adskilt fra selve projektstrukturen og kan være tilknyttet flere projekter via en junction-tabel “users_to_project”.
Strukturen sikrer en klar opdeling, hvor hvert niveau har ét entydigt tilhørsforhold, samtidig med at flere brugere kan samarbejde om de samme projekter.

Normal former
Alle tabeller har en tydelig primærnøgle, imens andre felter i tabellerne indeholder enkeltværdier.
Mange-til-mange relationen mellem users og project er håndteret gennem en junction tabel
Derved overholder vores design 1. NF.

Alle ikke nøgle-attributter afhænger af primærnøglen, imens at der ikke er nogen partielle afhængigheder. Junction-tabellen har en sammensat primærnøgle imens at der ikke er nogen dele af tabellen, der afhænger af kun den ene del.
Derved overholder vores design 2. NF.

Der er ingen transitive afhængigheder, ved at ikke nøgleelementer ikke er afhængige af et andet ikke nøgle element.

Derved overholder vores design 3. NF.



Vi har igennem projektet udarbejdet flere forskellige løsninger af ER diagrammet, samt databasens opbygning. Efterhånden som vi skred frem i processen af at lave projektet, lavede vi flere versioner af justerede ER-diagrammer som kan findes i [Bilag 3]

DDL

Det hierarkiske system er opbygget ved hjælp af "parent-child" relationer. Hvert underliggende element refererer til det overordnede element ved en fremmednøgle som er parent_id.

Et subprojekt refererer til et projekt ved at parent_id er projekt tabellens project_id. Tilsvarende referer en task til et subprojekt og en subtask til en task. Dette muliggør flere forgreninger der udspringer fra projekt til subprojekter, tasks og subtasks.

Strukturen håndhæver samtidig constraints, hvor subprojekt kun kan tilhøre ét projekt, en task kun kan tilhøre ét subprojekt og en subtask kun kan tilhøre én task. User er uafhængig

af den hierarkiske opbygning og kan eksistere uden tilknytning til projekter, ligesom projekter kan eksistere selvstændigt.
Vi har ligeledes integreret cascade regler i vores DML, der håndhæver vores struktur. Når et projekt slettes, så slettes tilhørende subprojekter. Tilsvarende bliver tasks slettet når tilhørende subprojekt bliver slettet og subtask ved sletning af en task.

```
CREATE DATABASE IF NOT EXISTS projectWizard;
USE projectWizard;


DROP TABLE IF EXISTS subtask;
DROP TABLE IF EXISTS task;
DROP TABLE IF EXISTS users_to_project;
DROP TABLE IF EXISTS sub_project;
DROP TABLE IF EXISTS project;
DROP TABLE IF EXISTS users;

CREATE TABLE users (
                    user_id INT AUTO_INCREMENT PRIMARY KEY,
                    username VARCHAR(255) NOT NULL UNIQUE,
                    email VARCHAR(255) NOT NULL UNIQUE,
                    user_password VARCHAR(255) NOT NULL
);


CREATE TABLE project (
                     project_id INT AUTO_INCREMENT PRIMARY KEY,
                     title VARCHAR(255) NOT NULL,
                     project_description TEXT,
                     created_at DATE,
                     deadline DATE
);

CREATE TABLE sub_project (
                        sub_project_id INT AUTO_INCREMENT PRIMARY KEY,
                        parent_id INT NOT NULL,
                        subproject_title VARCHAR(255) NOT NULL,
                        sub_project_description TEXT,
                        created_at DATE,
                        deadline DATE,
                        FOREIGN KEY (parent_id) REFERENCES project(project_id)
                           ON DELETE CASCADE
);

CREATE TABLE users_to_project (
                            user_id INT NOT NULL,
                            project_id INT NOT NULL,
                            PRIMARY KEY (user_id, project_id),

                            FOREIGN KEY (user_id) REFERENCES users(user_id)
```

```
                            ON DELETE CASCADE,

                        FOREIGN KEY (project_id) REFERENCES project(project_id)
                            ON DELETE CASCADE
);

CREATE TABLE task (
                task_id INT AUTO_INCREMENT PRIMARY KEY,
                title VARCHAR(255) NOT NULL,
                task_description TEXT,
                parent_id INT not null,
                task_status ENUM('NoStatus','InProgress','InReview','Done') DEFAULT
'NoStatus',
                estimated_time DOUBLE,
                assignee_id INT,
                actual_time DOUBLE,


                FOREIGN KEY (parent_id) REFERENCES sub_project(sub_project_id)
                    ON DELETE CASCADE,

                FOREIGN KEY (assignee_id) REFERENCES users(user_id)
                    ON DELETE SET NULL
);

CREATE TABLE subtask (
                    subtask_id INT AUTO_INCREMENT PRIMARY KEY,
                    title VARCHAR(255) NOT NULL,
                    subtask_description TEXT,
                    subtask_status ENUM('NoStatus','InProgress','InReview','Done')
DEFAULT 'NoStatus',
                    parent_id INT NOT NULL,
                    estimated_time DOUBLE,
                    assignee_id INT,
                    actual_time DOUBLE,

                    FOREIGN KEY (parent_id) REFERENCES task(task_id)
                        ON DELETE CASCADE,


                    FOREIGN KEY (assignee_id) REFERENCES users(user_id)
                        ON DELETE SET NULL
);
```

DML

Vores hierarkiske opbygning kan tydelig ses I vores DML da den følger hvornår databasens design tillader at indsætte hvilket data I hvilke tabeller.

```
INSERT INTO users(email, username, user_password)
VALUES ('adminEmail@email.com', 'admin', 'admin');

INSERT INTO project (title, project_description, created_at, deadline)
VALUES ('projectTestTitle', 'projectTestDescription','2024-01-01', '2025-01-01');

INSERT INTO sub_project(parent_id, subproject_title, sub_project_description,
created_at, deadline)
VALUES (1, 'subprojectTestTitle', 'subprojectTestDescription', '2024-01-01', '2025-
01-01');

INSERT INTO users_to_project(user_id, project_id)
VALUES (1, 1);

INSERT INTO task (title, task_description, parent_id,task_status, estimated_time,
assignee_id, actual_time)
VALUES ('taskTestTitle2', 'taskTestDescription2', 1, 'Done', 2.5, 1, 1.5);

INSERT INTO task (title, task_description, parent_id,task_status, estimated_time,
assignee_id, actual_time)
VALUES ('taskTestTitle', 'taskTestDescription', 1, 'NoStatus', 2.5, 1, 2.5);

INSERT INTO subtask(title, subtask_description, subtask_status, parent_id,
estimated_time, assignee_id, actual_time)
VALUES ('subtaskTestTitle', 'subtaskTestDescription',  'NoStatus', 1, 2.5, 1, 2.5);
```

Software Design – *(Andreas)*

Overordnet Arkitektur

Vores system er bygget op efter den klassiske trelagsarkitektur, hvor ansvaret tydeligt er fordelt mellem præsentationslaget, forretningslaget og dataadgangslaget. Bilag 6
Arkitekturen følger strukturen:

Controller → Service → Repository/DAO
For at overholde princippet Separation of Concerns[31] har vi benyttet os af denne struktur, som gør systemet mere overskueligt, lettere at teste og nemmere at vedligeholde.

UML Package Diagram

Diagrammet viser den overordnet pakkestruktur i vores system. Bilag 7

Controller-laget

Controller-pakken indeholder applikationens controller-klasser, som forbinder brugergrænsefladen til backend. Controlleren håndterer http-requests, modtager input fra brugeren, og afgør hvilke metoder der skal kaldes fra Service-laget. Controller-laget er afhængigt af service-laget.

Service-laget

Service-pakken indeholder applikationens service-klasser, hvor programmets forretningslogik ligger. De validerer data og bestemmer hvilke metoder der skal kaldes fra repository-laget. Service-laget fungerer som bindeled mellem controller- og repository-laget.

[31] https://www.geeksforgeeks.org/software-engineering/separation-of-concerns-soc/

Repository-laget

Repository-pakken er ansvarlig for dataadgang, og består af to underpakker, DAO og rowMapper, samt repository-klasserne. Opdelingen er valgt for at afskille den rene databaseadgang fra resten af repository-laget[Bilag 8] Det gør programmet mere overskuelig, og lettere at ændre hvis data-laget skulle ændres senere.
DAO-laget (Data Access Object) indeholder al logik relateret til CRUD-operationer og SQL-queries, og dermed al kommunikation til databasen.
RowMapper-laget indeholder implementeringer af Springs Framworks RowMapper-interface. Disse klasser er ansvarlige for at konvertere data fra databasen til Java-objekter i Model-laget.
Derudover indeholder pakken også repository-klasser, der fungerer som et bindeled mellem DAO og service-laget, og bidrager til en mere struktureret datahåndtering.

Model-laget

Model-pakken indeholder applikationens domæneklasser, såsom Project, Subproject, Task og Subtask. Klasserne er implementeret som POJOs[32] og indeholder kun konstruktører, getters og setters, uden yderligere logik.

Exceptions

Systemet indeholder desuden en expeptions-pakke, der dog ikke er med i UML package-diagrammet. Denne pakke indeholder brugerdefinerede exceptions. Disse bruges til en til at give en præcis fejlhåndtering af fejlen på en forståelig måde.

Class Diagram

Med udgangspunkt i vores Class Diagram [Bilag 8] kan vi tage konkret eksempel på, hvordan en http-request bevæger sig igennem applikations arkitektur, når en bruger opretter et projekt. Den samme struktur anvendes for Subproject, Task og Subtask, men for overskuelighedens skyld tages der i dette udgangspunkt i projekt-flowet.

Når en bruger trykker på ”Create” sendes der en HTTP POST request til endpoint ”/createProject”. Den håndteres af controlleren af denne metode:

[32] https://en.wikipedia.org/wiki/Plain_old_Java_object

```
@PostMapping("/createProject")
public String createProject(
        @RequestParam String title,
        @RequestParam String description,
        @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
LocalDate deadline,
        Model model) {

    user = loginService.checkIfLoggedInAndGetUser(session);

    Project project = new Project(title, description, deadline);

    try {
        projectService.createProjectWithCreator(project, user.getUserId());
        model.addAttribute("message", "Project created successfully!");
        return "redirect:/project/home";

    } catch (RuntimeException e) {
        e.printStackTrace();
        model.addAttribute("error", "Could not create project. Please try
again.");
        return "createProject";
    }
}
```

Controlleren modtager brugerens input ved @RequestParam. Før projektet oprettes, kontrolleres det om brugeren er logget ind.

Der tjekkes om brugeren er logget ind, ved at anvende loginService hjælpemetode checkIfLoggedInAndGetUser med session i inputparameter.

```
public User checkIfLoggedInAndGetUser(HttpSession session) {
    User user = (User) session.getAttribute("loggedInUser");
    if (user == null) {
        throw new UserNotLoggedInException("Please log in to continue");
    }
    return user;
}
```

Hvis brugeren ikke findes, kaster systemet en exception, og create project bliver afbrudt.

Applikationen bruger http-sessions til at gemme hvilke bruger der er logget ind. Brugerobjektet gemmes i session under attributten ”loggedInUser”.

```
session.getAttribute("loggedInUser");
```

Applikationen håndterer exceptions ved try-catch eller custom exceptions. Eksempel, når en bruger ikke er logget ind:

```
throw new UserNotLoggedInException("Please log in to continue");
```

Efter brugeren er valideret sendes projektet til service-laget.

```
@Transactional
public void createProjectWithCreator(Project project, int userId) {
    projectRepository.createNewProject(project);
    int projectId = projectRepository.getLastCreatedProjectId();
```

```
    projectRepository.addUserToProject(userId, projectId);
}
```

Denne metode er annoteret med @Transactional. Dette betyder at oprettelse af projekt og tilknytning til bruger behandles som en transaktion. Så hvis en del fejler, rulles hele metoden tilbage.

Det sendes nu videre til repository-laget.

```
@Repository
public class ProjectRepository {

    private final ProjectDAO projectDAO;

    public ProjectRepository(ProjectDAO projectDAO) {
        this.projectDAO = projectDAO;
    }

    public int createNewProject(Project newProject) {
        return projectDAO.createNewProject(newProject);
    }

    public int getLastCreatedProjectId() {
        return projectDAO.getLastCreatedProjectId();
    }

    public int addUserToProject(int userId, int projectId) {
        return projectDAO.addUserToProject(userId, projectId);
    }
}
```

Repository-laget videresender til projectDAO-laget.

DAO-laget kommunikerer med databasen via JdbcTemplate.
Oprettelsen af det nye projekt sker ved følgende SQL-query:

```
@Transactional
public int createNewProject(Project newProject) {
    String sql = "INSERT INTO project (title, project_description,
created_at, deadline) VALUES (?, ?, ?, ?)";
    return jdbc.update(
            sql,
            newProject.getTitle(),
            newProject.getDescription(),
            LocalDate.now(),
            newProject.getDeadline()
    );
}
```

Derefter hentes det ID på det seneste oprettede projekt:

```
public int getLastCreatedProjectId() {
    String sql = "SELECT MAX(project_id) FROM project";
    return jdbc.queryForObject(sql, Integer.class);
}
```

Til sidst oprettes en relation mellem den bruger og projekt:

```
public int addUserToProject(int userId, int projectId) {
```

```
    String sql = "INSERT INTO users_to_project (user_id, project_id) VALUES
(?, ?)";
    return jdbc.update(sql, userId, projectId);
}
```

Dette sikrer at brugeren bliver tilknyttet til det projekt som de selv har oprettet.

Hvis alt dette er gået igennem uden fejl, returneres flowet tilbage til controlleren som redirecter til projektets forside:

```
return "redirect:/project/home";
```

Test og Kodeanalyse – *(Andreas)*

Formålet med testarbejdet i vores projekt har været at sikre, at systemet funktionalitet fungerer. Vi har benyttet os af tre typer af test for at opnå et bredt og balanceret dækningsniveau:

- Unit tests
  Fokus på at teste individuelle metoder og komponenter isoleret, primært i service-laget.

- Integrationstests
  Tester sammenspillet mellem lag i applikationen, herunder repository-laget mod en h2 database.

- Web layer slice Tests
  Tester controlleren returnerer rigtige views, håndtere brugerinput og samarbejdet med service-laget.

Kombinationen af alle tests har givet os et solidt overblik over applikationens stabilitet og funktionalitet.

Oversigt over testede klasser og metoder:

Lag	Klasse	Metoder der er testet	Testtype
Controller	LoginController	POST/login	WebLayerSliceTest
Controller	ProjectController	GET /project/home, redirection, model attributes	WebLayerSliceTest
Controller	TaskController	GET/POST (create task + subtask)	WebLayerSliceTest
Controller	UserController	POST /users/register	WebLayerSliceTest
Service	ProjectService	createProjectWithCreator()	UnitTest
Service	StatisticsService	11 statistik-metoder	Integrationtest
Repository	ProjectRepository	CRUD + join metoder	Integrationtest h2
Repository	UserRepository	createNewUser(), getUserByEmail()	Integrationtest

Repository	SubprojectRepository, TaskRepository, SubtaskRepository	Cascade deletion	Integrationtest

Kodeanalyse med Qodana

I projektet har vi brugt Qodana til statisk kodeanalyse. Værktøjet har kørt automatisk gennem GitHub Actions i forbindelse med pull requests og merges til main branchen. Formålet har været at fange potentielle fejl løbende, så de kunne rettes, før koden blev lagt i main. Continuous Integration & Deployment/Delivery – *(Ella)*

Som set på[Bilag 9] identificerede Qodana 28 problemer i alt.

- 25 problemer relaterer sig til Spring MVC – Unresolved View Reference. Disse problemer opstår fordi Qodana ikke kan finde de HMTL-views som controlleren returnerer. Men da views fungere når applikations kører, er advarslerne vurderet som falsk positiver, og bliver ikke rettet.

- 2 problemer i kategorien Code blev identificeret og rettet inden merge til main.

- 1 problem i kategorien Probable bug og omhandler en NullPointerException i ProjectDAO. Dette problem opstår fordi Qodana mistænker en NullPointerException, som vi har forsøgt at løse ved at throw NullPointerException, men Qodana tror foresat det er en fejl. Vi mistænker at den refererer til NullPointerException et andet sted i koden, end der vi har håndteret det.

Alt i alt har Qodana være nyttig, da den gjorde det nemmere at opdage fejl tidligt og holde koden mere ensartet.

Scrumprocessen – *(Ella)*

Vi har i dette projekt arbejdet ud fra Scrum[33], som er en agil udviklingsmetode[34] der fremmer effektivitet og struktur under projektarbejde. Nedenfor beskrives først arbejdsprocessen faktuelt og dernæst beskrives den reflekteret.

Arbejdsprocessen faktuelt

For at skabe struktur fastlagde vi selv et skema som vi fulgte under hele projektets varighed, det kan ses herunder:

	Mandag	Tirsdag	Onsdag	Torsdag	Fredag
9.30-9.45	*Sprint Start møde*	*Stand Up*	*Stand Up*	*Stand Up*	*Stand Up*
10.15-10.30	*Sprint Start møde-fortsat*			PO-møde (David)	
10.45-11.00			*Vejledning*		
11.00-12.00					
12.00-12.45	*Frokost*	*Frokost*	*Frokost*	*Frokost*	*Frokost*
12.45-14.00					*Sprint slut-møde*
14.00-14.30	PRs og Reviews	PRs og Reviews	PRs og Reviews	PRs og Reviews	PRs og Reviews

Daily stand-up

Her opdaterede vi hinanden ved at besvare 3 spørgsmål:

1. Hvad lavede du igår?
2. Hvad skal du lave i dag?
3. Har du brug for hjælp med noget?

På disse stand-up møder fik vi et hurtigt indblik i, hvad de resterende Team members var i gang med. Desuden kunne vi stille spørgsmål og besvare andres spørgsmål. Det hjalp os med at bevare oplevelsen af at arbejde i gruppe, selvom vi brugte meget tid på opgaverne individuelt.

Sprint Planning

[33] https://www.scrum.org/resources/what-scrum-module
[34] https://en.wikipedia.org/wiki/Agile_software_development

Hver mandag afholdte vi et sprint-start møde hvor vi baseret på de forhenværende review- og retrospektiv-møder kunne sætte dagsordenen for det kommende sprint. Vi har valgt at bruge vores opstarts-møde af Sprint 3 som eksempel:

> *Mandag d. 8/12*
> **Ugens Scrum Master:** *Theis*
> **Noter:**
> *"Vi skal påbegynde et større oprydningsarbejde i vores kode, desuden skal dashboard færdiggøres og linkes op til resten af projektet, så vi kan påbegynde tjek for eventuelt oversete features – og implementere dem hvis der er mangler –, samt rette i nuværende fejl eller bugs der måtte fremgå i koden eller andre steder i projektet"*
> **Mål for næste sprint:**
> - *Man skal kunne se, tilføje og fjerne medlemmer i dashboard*
> - *Man skal kunne se assignees visuelt på den task eller subtask de er assignet til.*
> - *Man skal kunne se edits på subtask generelt*
> - *Tilbage knap skal tilføjes på alle relevante sider og navigere korrekt tilbage*
> - *Gennemtjek programmet for bugs, fejl og mangler – fjern/ret/tilføj*

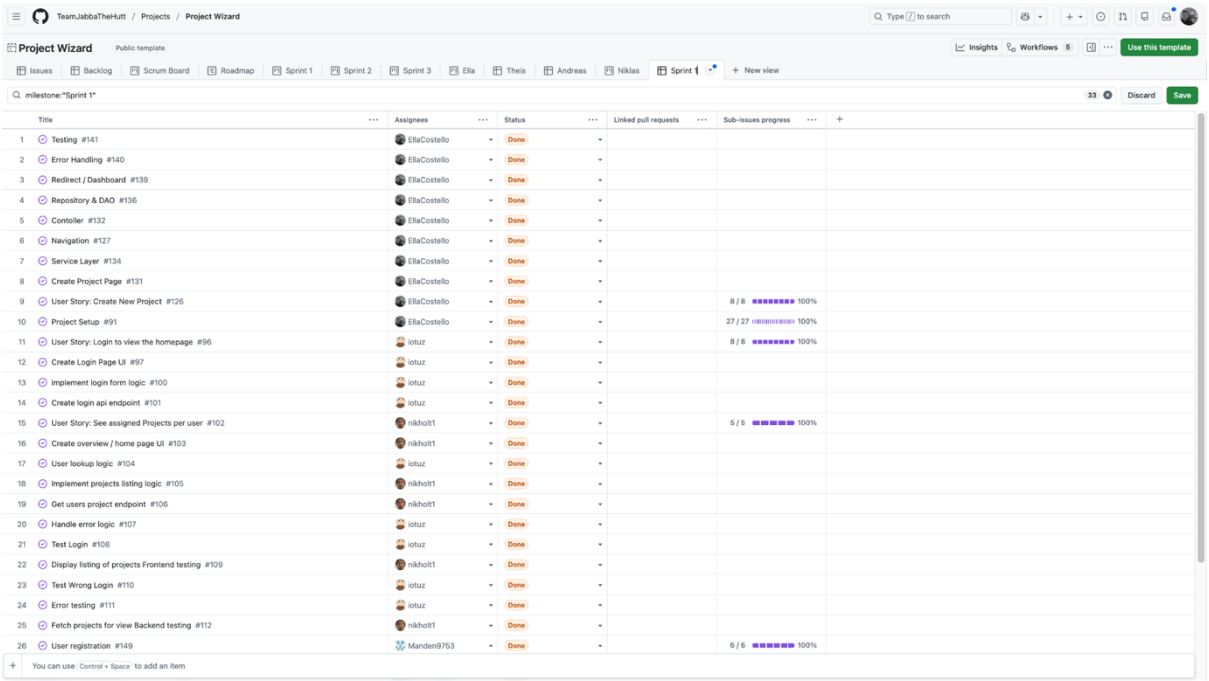
Med en mindre overskredet deadline *(2 dage, færdig søndag aften)* nåede vi at opfylde alle krav og mål for Sprint 3.

Sprint Review m. Product Owner (PO)
Hver torsdag afholdte vi check-in og sprint review med vores PO, her diskuterede vi prioriteringer, hvordan sprintet var gået og lavede review på de opgaver der var lavet i overensstemmelse med sprint-backloggen. Vi har valgt at bruge vores review-møde af Sprint 1 som eksempel:

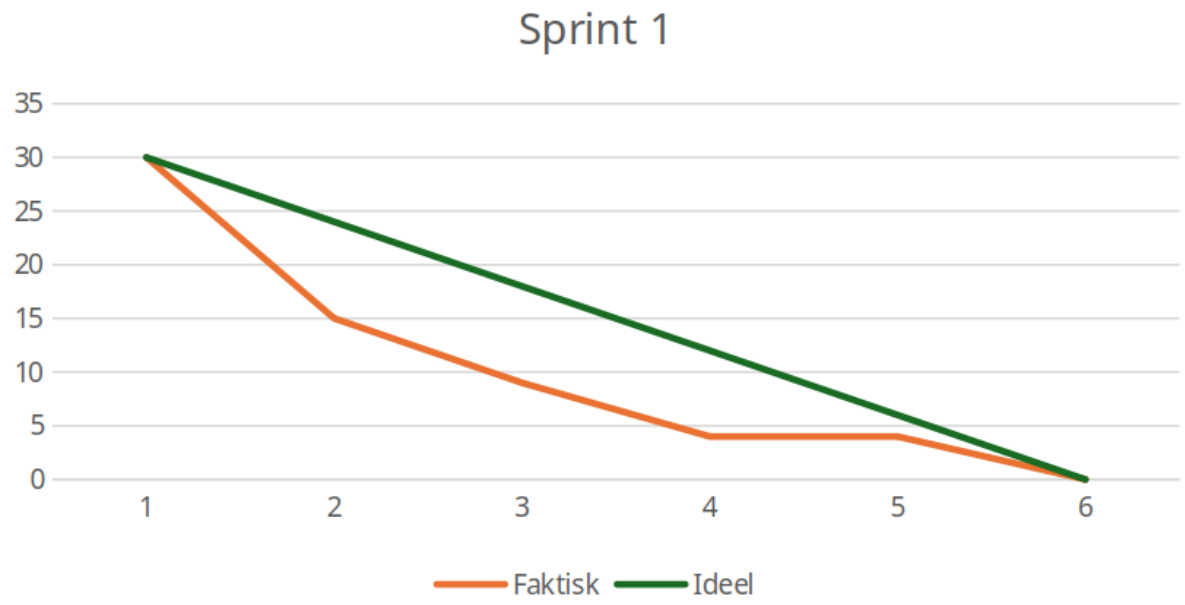
> *Torsdag d. 27/11*
> **Noter:**
> *"Vi foreslog til PO at tilføje en ekstra user story til sprintets backlog da vi har tidsestimeret forkert og faktisk har flere timer i et sprint end vi havde udregnet, og derfor ikke har fyldt projektet ud med det korrekte antal opgaver. PO informerer os om at et sprint-backlog er 'frosset' når først det er planlagt. Vi forsøger at finde ud af, om vi kan påbegynde sprint 2 før tid, og aftaler at fokuserer på UI/UX +*

bugfixing/refinement indtil vi ved om vi kan lave opstart før tid."

Under opstart til sprint 1 havde vi lavet en udregning på hvor mange mandetimer vi havde på et sprint, pauser, møder og diverse taget i betragtning, denne udregning viste sig at være forkert og vi lagde derfor ikke nok tasks ind på sprintets backlog (se billede), dette medførte at vi hurtigt stod uden tasks. Vi valgte at bruge sprintet på refinement, da det ellers ville bryde med vores skema, det var en fælles vurdering med PO og vi var tilfredse med resultatet.



Vores mix-up resulterede I at vi hele tiden var hurtigere end forventet, hvilket kan ses på dette burndown-chart af sprint 1:

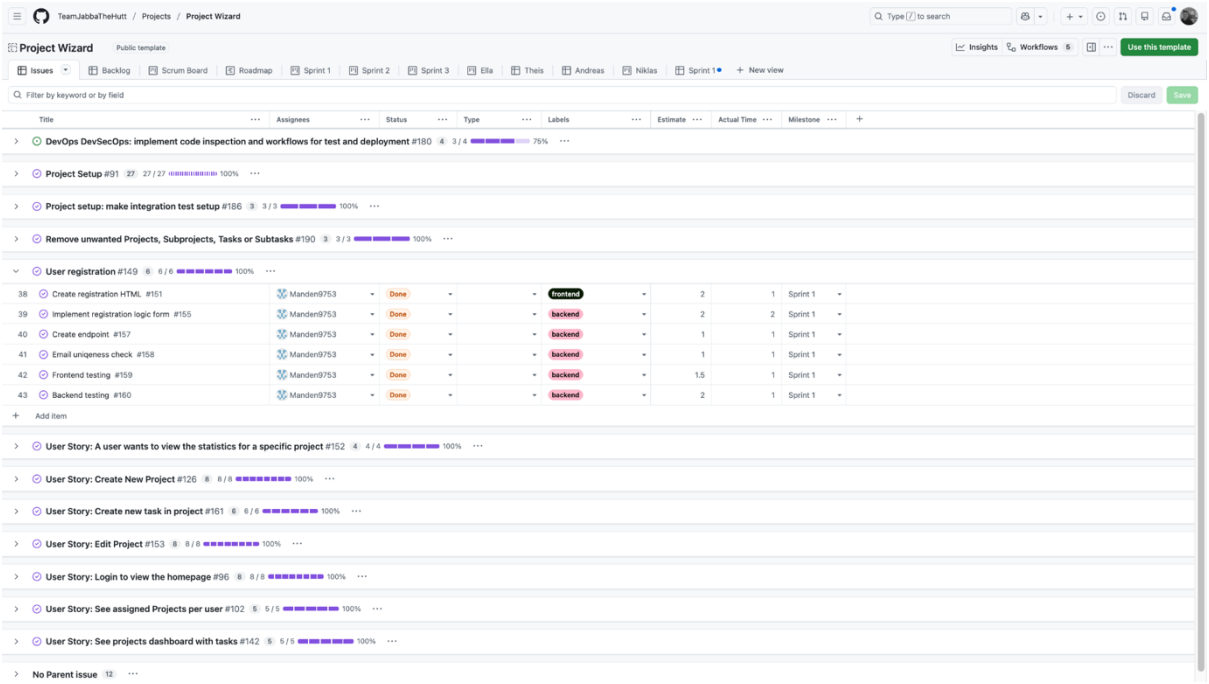


Sprint Retrospective

Hver fredag holdte vi retrospective for internt review af sprintet, tjek for overholdelse af deadlines og evaluering generelt. Vi brugte møderne på at vurdere hvilke aspekter der var gået godt, og hvilke der var gået mindre godt, vi fik gennemgået de andre gruppemedlemmers arbejde og kunne derfor føle ejerskab over projektet som helhed. Til hvert retrospektiv blev der diskuteret kravspecifikation, og om vi stadig var motiverede, glade og tilfredse med gruppens samarbejde.

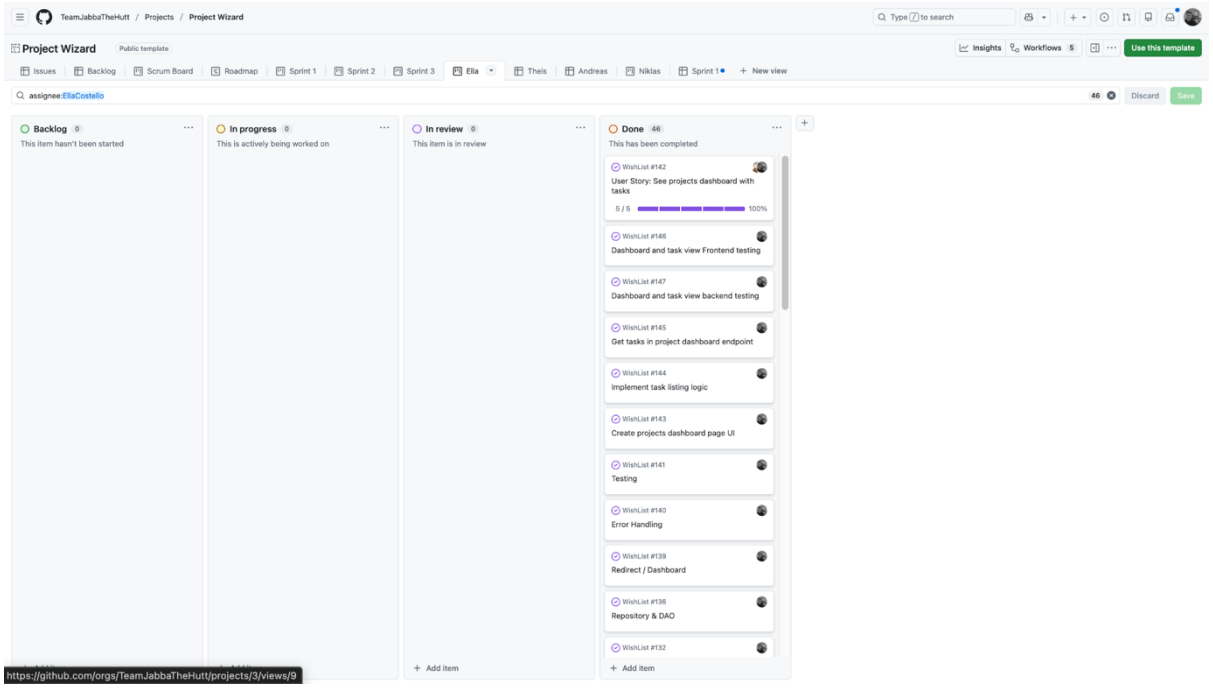
Vores design og opsætning af Github Projects

Som udgangspunkt udfyldte vi Product backloggen med alle de user-stories og tasks der skulle laves for at opnå det produkt vi ønskede. Herunder kan vores product backlog ses (med en udfoldet user story som eksempel):

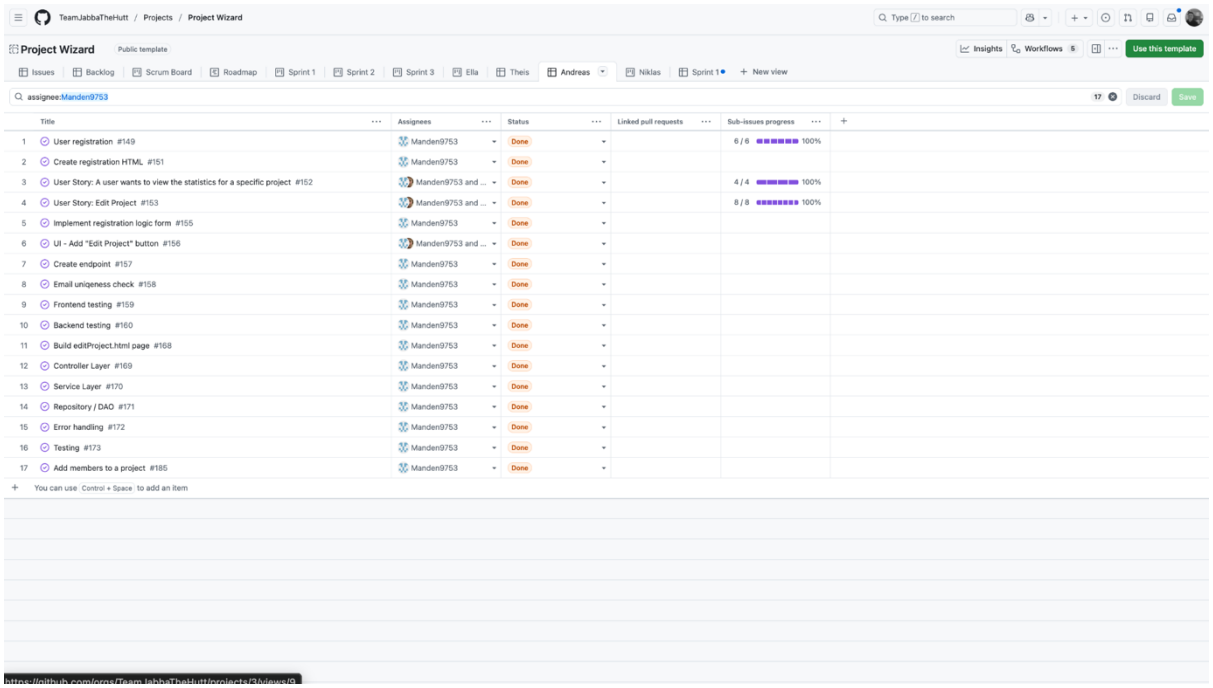


Derudover oprettede vi forskellige faner, der gav os mulighed for individuelt at få et mere specifikt overblik over issues. Vi vil her gerne påpege vores gruppemedlems-specifikke views, hvor hvert medlem kan få vist et view i enten scrum board eller backlog format, de views er filtreret på, at pågældende bruger er sat på som assignee. Herunder vises to eksempler hvor gruppemedlemmer har valgt et forskelligt design for at se de issues hvor de står som assignee.

Eksempel 1: Ella (Scrum board view)



Eksempel 2: Andreas (Backlog view)



Arbejdsprocessen reflekteret

I dette afsnit vil vi reflektere over hvilke dele der er gået godt i forbindelse med Scrum, og hvilke der er gået mindre godt.

Dét, der er gået godt:

- **Scrum Master rollen** – ugentlige udskiftninger gav ¾ gruppemedlemmer mulighed for at sidde i en anderledes rolle end de er vant til. Det gør også at alle kan få lov til at være ansvarshavende, ikke blot for programmeringsdelen, men også for systemudviklings-delen.

- **Tidsestimering af User stories & Tasks** – vi benyttede planning poker[35] til at tidsestimere. Vi benyttede spændet 1-10 timer og vi var i store træk enige, denne metode var vi glade for, da nogle gruppemedlemmers kompetencer måske ligger et andet sted, end de andres. Derfor kunne estimatet vurderes på et samlet udgangspunkt.

- **Nedbrydning af User stories og tasks** – Vi hjalp alle med at nedbryde projektet, så vi havde alle krav og forventninger med i backloggen[Bilag 10] Vi havde desuden designet pakke strukturen og databasen forinden, så vi vidste præcis, hvilke delopgaver skulle placeres hvor henne i projektet.

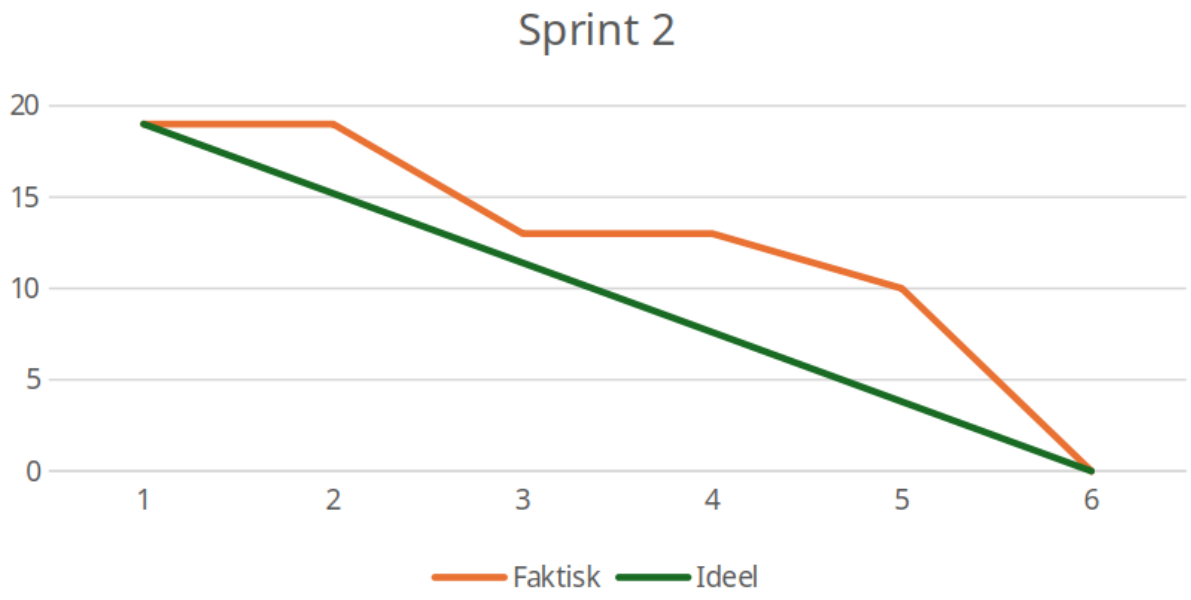
Dét, der er gået mindre godt:

- **Estimering af rådighedstimer per Sprint** – Som beskrevet foroven havde vi udregnet vores mandetimer forkert, og det resulterede i et Sprint 1, hvor der kunne have ligget flere user stories end der gjorde.

- **Opdeling af user stories per gruppemedlem** – I det første sprint havde vi ikke gennemtænkt fordelingen af user stories, det resulterede i, at én af medlemmerne havde en user story der var vurderet til 2~3 gange så mange timer som de andre medlemmers, dette rettede vi dog op på i efterfølgende sprint.

Som tilføjelse til det der er gået mindre godt, havde vi i gruppen under sprint 2, i nogle tilfælde glemt at sætte tasks til "Done" det resulterede i et burndown chart der ikke helt afspejler virkeligheden, men vi har lært af fejlen og synes det stadig det skal medbringes:

[35] https://planningpokeronline.com/


Sprint 2
20
15
10
5
0
1
2
3
4
5
6
Faktisk
Ideel

Continuous Integration & Deployment/Delivery – *(Ella)*

Når man arbejder med den samme kode, men fra 4 forskellige enheder, kan der hurtigt opstå problemer og uoverensstemmelser. Vi kan komme til at omskrive hinandens kode, slette ændringer de andre har lavet, listen af potentielle problemer er lang. For at håndtere den problematik har vi implementeret en CI/CD pipeline, der gør at vi aldrig sidder direkte i produktionsmiljøet, i stedet sidder vi i forskellige udviklingsmiljøer og opretter pull-requests til produktions-koden. Ved hjælp af nogle automatiserings værktøjer kan vi opfange de fejl og problemer der måske følger med de ændringer vi forsøger at skubbe til vores produktions miljø. Herunder beskriver vi mere dybdegående vores proces:

Værktøjer:

Vi anvender GitHub til versionsstyring og GitHub Actions til automatisering af build-, test- og deploy-processen. Projektet er skrevet i Java og bygges med Maven. Derudover anvender vi Qodana til at lave statisk kodeanalyse, det hjælper med at kvalitetssikre koden samt finde og identificere potentielle problemer. Til deployment bruges en Azure Web App ressource hvor projektet deployes så programmet kan tilgås online.

Branching Strategi:

For ikke at miste vores 'One source of truth'-kildekode, har vi valgt, at alle tilføjelser til main-branchen sker gennem pull requests. Når vi skal i gang med en ny programdel eller opdatering, opretter vi en feature-branch, hvor vi til alle rettelser på eksisterende kode-dele bruger bug-branches. Når branchen vurderes at være klar til at komme i produktion (main), oprettes en pull request og vores CI/CD-pipeline går i gang.

Pipeline:

CI/CD-pipelinen er implementeret via flere GitHub workflows:

- **Continuous Integration (CI) - *Java CI with maven***
  Workflowet køres automatisk ved oprettelse af en pull request eller ved push til main. Workflowet opsætter JDK 21, cacher alle Maven-dependencies og kører

kommandoen ‘mvn package’ for at bygge projektet. Workflowet sikrer at vi ikke støder ind i compiler-fejl og at nye ændringer ikke introducerer fejl i buildet. Skulle fejl i workflowet opstå vil vi kunne identificere de problemer der ligger i koden, lave ændringerne og genstarte workflowet før vi pusher det ind i main, for at sikre at main forbliver robust.

- **Kvalitetssikring af kode - *Qodana***
  Workflowet for Qodana instantieres på samme tid som maven-workflowet. Det sørger for at vi -ligesom med forrige workflow- kan lave de nødvendige rettelser før koden bliver sendt i produktion.

- **Continuous Deployment/Delivery - *Build and deploy JAR app to Azure Web App***
  Ved push til main bygges projektet og den genererede JAR-fil deployes automatisk til en Azure Web App-ressource som vi har oprettet. Deployment-processen bruger secrets til at forbinde GitHub Actions til Azure ressourcen.

Bemærkninger til workflows

På nuværende tidspunkt kører workflows Java CI with Maven & Qodana ikke kun på pull-requests, men også på push til main. Det er overflødigt at den også kører i push til main i henhold til vores branching strategi.

Vi er ikke sikre på hvordan, men i det vi pusher til main og starter Build and deploy workflowet, burde vi kunne tilbagerulle pull requesten hvis workflowet fejler. Uanset hvad resultatet af workflowet er, som det ser ud lige nu, vil pull requesten blive merget ind i main.

Status på implementering – *(Ella)*

Her vil vi beskrive funktioner vi enten ikke har nået at implementere, eller som blev ved designfasen.

- At give brugere mulighed for at redigere deres oplysninger – herunder e-mail og/eller brugernavn.

- Udvikle funktionalitet og logik til Forgot your Password knappen der ligger i login – lige nu navigerer knappen til samme side (login).

- My Assignments view – Her vil en bruger få et samlet overblik over alle de subprojects, tasks & subtasks hvor de er markeret som Assignee. Dette gælder på tværs af alle projekter hvor de er på listen af medlemmer.

- Responsivt design – Ligenu er store dele af vores styling lavet med faste størrelser (px), dette gør at applikationen ikke vil være brugbar cross-platform, og også vil kræve en hvis størrelse på det vindue hvor applikationen er åben.

- Specifikke deadlines på tasks og subtasks – Dato for deadline arves på nuværende tidspunkt fra det subproject hvor de hører til.

- Flere Assignees – Kun en Assignee kan tilføjes på en opgave, hvis man tilføjer en ny Assignee, vil den forhenværende blive fjernet.

Acceptance Test

Med udgangs punkt I vores User Stories[Kravspecifikation – User Stories – *(Theis)*,] Har vi lavet en række acceptance tests, de er lavet baseret på oprindelige acceptkriterier.

Acceptance test 1 - US#96

	Acceptkriterier	**Status**
1.1	Brugeren kan ind	✅
1.2	Systemet validere login-oplysninger	✅
1.3	Ved korrekt login viderestilles brugeren til home page	✅
1.4	Ved forkert login vises en fejlmeddelelse	✅

Acceptance test 2 - US#102

	Acceptkriterier	**Status**
2.1	Brugeren kan få vist en projektoversigt efter login	✅
2.2	Systemet viser kun projekter, som brugeren er tilknyttet	✅
2.3	Hvis brugeren ikke er medlem af nogen projekter, vises en passende besked	✅

Acceptance test 3 - US#126

	Acceptkriterier	**Status**
3.1	Brugeren kan tilgå en “Create New Project-formular	✅
3.2	Formularen indeholder: Projekttitel, beskrivelse, deadline	✅
3.3	Systemet validerer input ved oprettelse	✅

3.4	Projektet gemmes I databasen ved gyldig input	✅
3.4	Brugeren viderestilles til homepage hvor projekterne kan ses	✅

Acceptance test 4 - US#149

	Acceptkriterier	**Status**
4.1	Brugeren kan indtaste navn, email og adgangskode	✅
4.2	Systemet validerer, at email ikke allerede eksisterer	✅
4.3	Ved gyldig input oprettes brugeren i systemet	✅
4.4	Brugeren kan efterfølgende logge ind	✅
4.5	Ved fejl vises en passende fejlmeddelelse	✅

Acceptance test 5 - US#142

	Acceptkriterier	**Status**
5.1	Trykker på projektet og bliver navigeret til dette projekts dashboard	✅
5.2	Alle brugerens subprojects og tasks/subtasks vises som klikbare elementer der udfolder sig, så mere information bliver givet.	✅

Acceptance test 6 - US#153

	Acceptkriterier	**Status**
6.1	Brugeren kan redigere projekttitel, beskrivelse og deadline	✅
6.2	Brugeren kan vælge at ændre medlemmere	✅
6.3	Brugeren kan indtaste email på et medlem der ønskes fjernet	✅
6.4	Systemet validerer om brugeren eksisterer	✅
6.5	Medlemmet fjernes fra projektet ved succes	✅
6.6	Ved ugyldig email vises en fejlmeddelelse	✅

Acceptance test 7 - US#161

	Acceptkriterier	**Status**
7.1	Brugeren kan angive: - Titel - Beskrivelse	✅
7.2	Som gemmes i databasen og bliver vist under project	✅

Acceptance test 8 - US#152

	Acceptkriterier	**Status**
8.1	Statistikside viser totale estimerede timer, faktiske timer, resterende timer.	✅
8.2	Viser antal opgaver færdige vs. total.	✅
8.3	Viser % fuldført baseret på tasks.	✅
8.4	Statistikker opdateres dynamisk efter databasens data.	✅

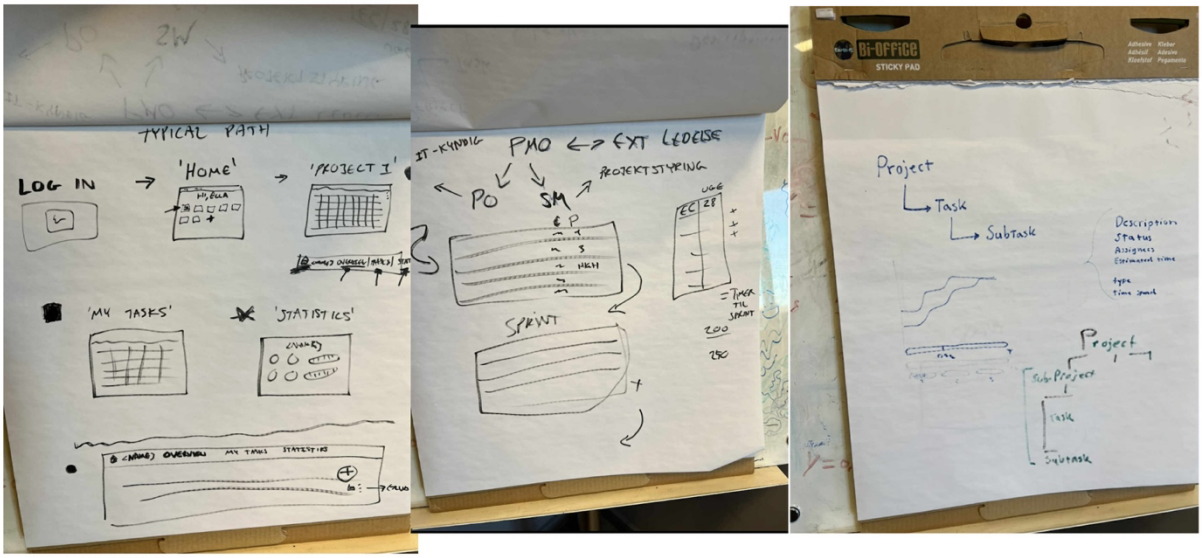
Konklusion

Formålet med denne opgave har været at udvikle et projektkalkulationsværktøj til Alpha Solutions. De beskrev et behov for et program, der hjælper dem med overblik, struktur og tidsestimering. Gennem vores arbejde med User Stories, virksomhedsanalyser (herunder feasibility study, interessentanalyse og riskioanalyse) identificerede vi de funktionelle krav til at understøtte virksomhedens opgave. Dette sikrede at løsningen tog udgangspunkt i deres behov frem for tekniske antagelser. Domænemodel og database er konstrueret med et naturligt hierarki, som aftegner hvordan det ser ud i virkeligheden. Ved hjælp af grundlæggende principper indenfor UX/UI har vi sørget for at give brugeren et brugervenligt overblik over projektet, og de redskaber der skal til for at vedligeholde et projekt. Vi har dokumenteret og arbejdet ud fra SCRUM-principperne, som har gjort vores projektarbejde mere effektivt og robust. Vi har implementeret flere lag af tests til vores kilde kode, og implementeret en CI/CD pipeline, der automatiserer processer som er kritiske for projektets succes. Vi er stolte og tilfredse med vores endelige produkt, da det opfylder alle kravene vi fik stillet, og tildels også stillede os selv. Processen har været lærerig for os alle, da vi har forbedret vores kompetencer indenfor programmering, systemudvikling, teknologi, agilt projektarbejde og UX/UI- design.

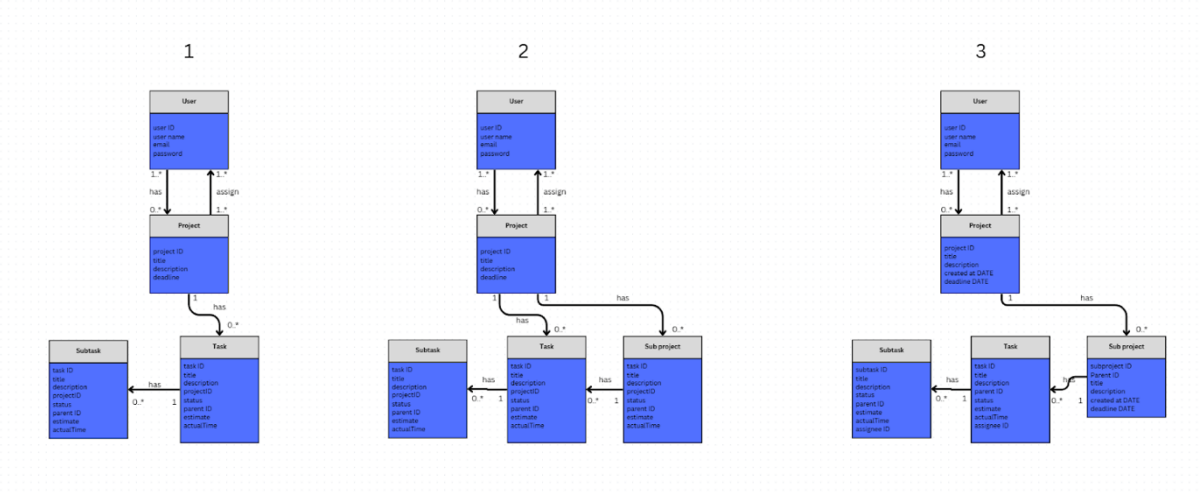
Litteraturliste

[1] https://azure.microsoft.com/en-us/pricing/details/app-service/static/
[2] https://adsense.google.com/start/ - calculator
[3] https://github.com/TeamJabbaTheHutt/ProjectWizard/blob/main/README.MD
[4] https://www.jetbrains.com/idea/
[5] https://www.apache.org/licenses/LICENSE-2.0
[6] https://www.java.com/en/
[7] https://www.gnu.org/licenses/old-licenses/gpl-2.0.en.html
[8] https://www.mysql.com/
[9] https://maven.apache.org/
[10] https://spring.io/projects/spring-boot
[11] https://html.spec.whatwg.org/multipage/
[12] https://www.w3.org/Style/CSS/
[13] https://mvnrepository.com/artifact/mysql/mysql-connector-java
[14] https://spdx.org/licenses/LGPL-2.1-only.html
[15] https://h2database.com/html/build.html?highlight=maven&search=maven - firstFound
[16] https://www.mozilla.org/en-US/MPL/2.0/
[17] https://www.jetbrains.com/qodana/
[18] https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows
[19] https://github.com/actions/checkout
[20] https://opensource.org/license/mit
[21] https://github.com/actions/setup-java
[22] https://azure.microsoft.com/en-us/free/students
[23] https://www.designprinciplesftw.com/collections/10-usability-heuristics-for-user-interface-design
[24] https://www.designprinciplesftw.com/collections/shneidermans-eight-golden-rules-of-interface-design
[25] https://www.interaction-design.org/literature/topics/gestalt-principles?srsltid=AfmBOop-QOCBP9FbYwBv95KvMjXcg7GCD--ZioluFGcpm1-R0-ZkEIm8
[26] https://github.com/EK-DATA-2SEM-PROGSYSTEK/DATA-GBG-F25A-B/blob/master/Uge43/2.gang/gestaltlove.jpg
[27] https://www.canva.com/
[28] https://github.com/EK-DATA-2SEM-PROGSYSTEK/DATA-GBG-F25A-B/blob/master/Uge43/2.gang/Design-thinking-101-NNG.pdf
[29] https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/
[30] https://media.nngroup.com/media/articles/attachments/Heuristic_Evaluation_Workbook_1_Fillable.pdf
[32] https://www.geeksforgeeks.org/software-engineering/separation-of-concerns-soc/
[33] https://en.wikipedia.org/wiki/Plain_old_Java_object
[34] https://www.scrum.org/resources/what-scrum-module
[35] https://en.wikipedia.org/wiki/Agile_software_development
[36] https://planningpokeronline.com/

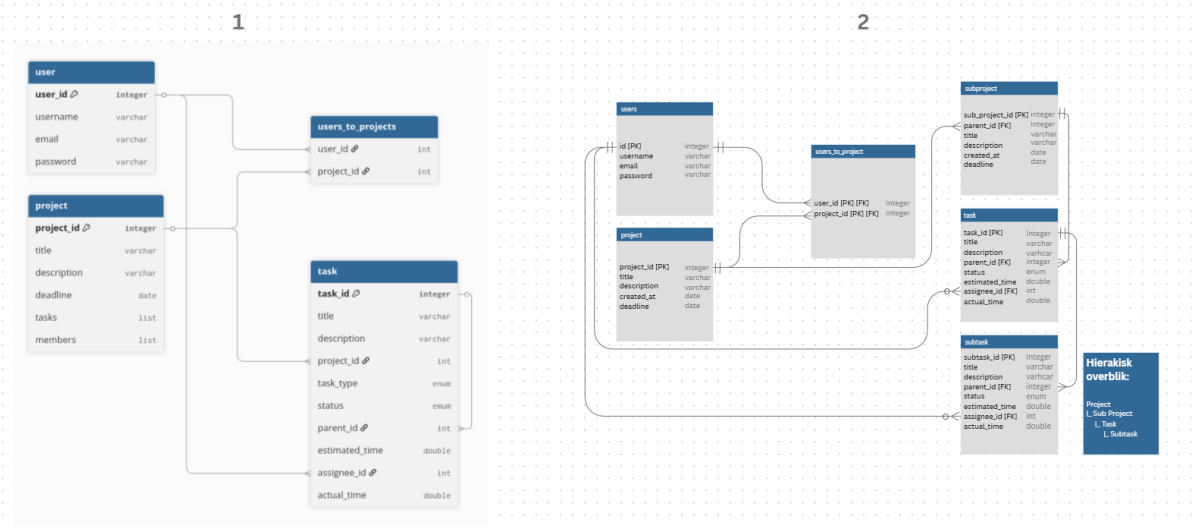
Bilag



Bilag 1



Bilag 2



Bilag 3

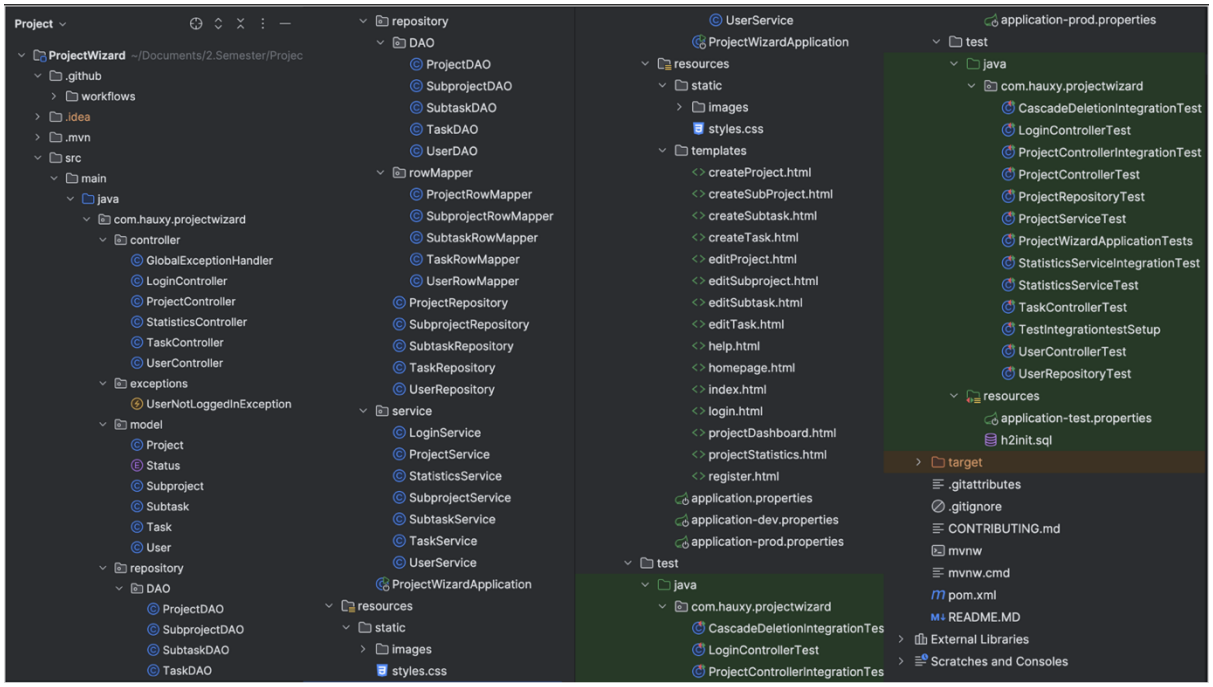
Kategori	Problemstilling	Optimeringstiltag	Effekt(1-10)	Omkostning(1-10)	Prioritering
Dashboard	Flere participants kunne ikke se at der var mulighed for at droppe / folde subprojects, tasks og subtasks ned	Implementerer en pil der indikerer muligheden for drop down, der ændrer sig i forhold til om table er droppet / foldet ud eller ej	7	1	8
Edit task og subtask status	Flere participants efterspurgte en større indikator på at Status kunne foldes ud	Implementer en pil der indikerer muligheden for drop down, der ændrer sig i forhold til om table er foldet ud eller ej	4	1	5
Edit task og subtask status	Farve på edit gjorde at flere af mulighederne for at ændre ting svært at finde, da de lettere blev overset	Større kontrast farver eller brug af mørkere eller lysere farve kontrast til at indikere at der er et interaktivt felt	4	1	5
Statistics	Participants havde problemer med at navigere tilbage til det tilhørende project efter at have set statistics	Implementer en knap til at returnere tilbage til det projekt man ser statestikkerne på	5	2	7
Landing page	Knap til "go to login" mangler	implementer "go to login" knap på landing page der	3	1	4

		tager user til login page			
Dato vælger	Flere participants trykker på tallene for at få kaldender for deadline frem, og opdager ikke at kalender knappen er i højre side	Implementer navn over input feltet og eventuelt en besked	7	3	10
Time estimate og time actual på elementer	Ved fejl kom brugeren til at sætte estimate til at være tom og trykkede enter, dette resulterede i en systemfejl der crashede programmet.	implementer fix der sørger for at hvis feltet er tomt, bliver det automatisk sat til 0.0	9	4	13

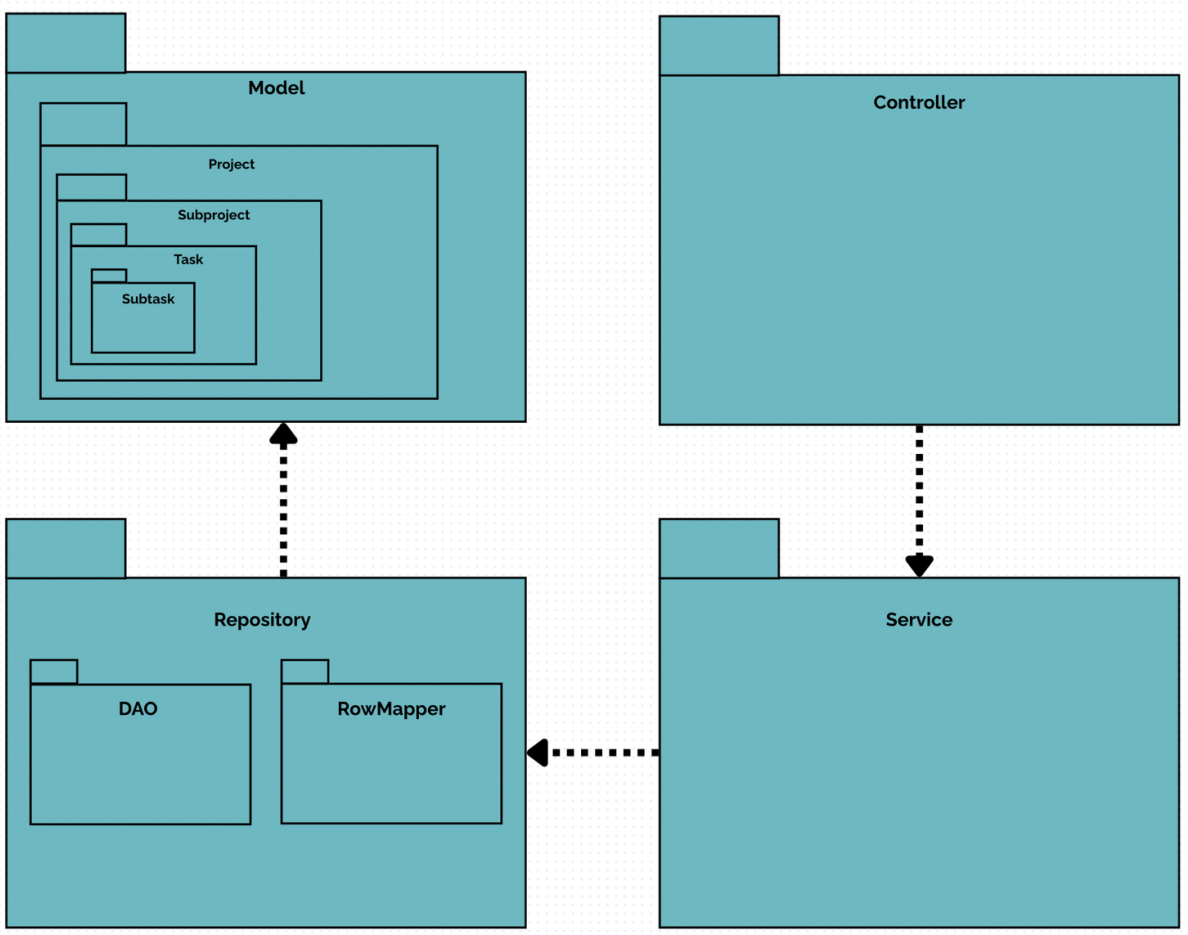
Bilag 4



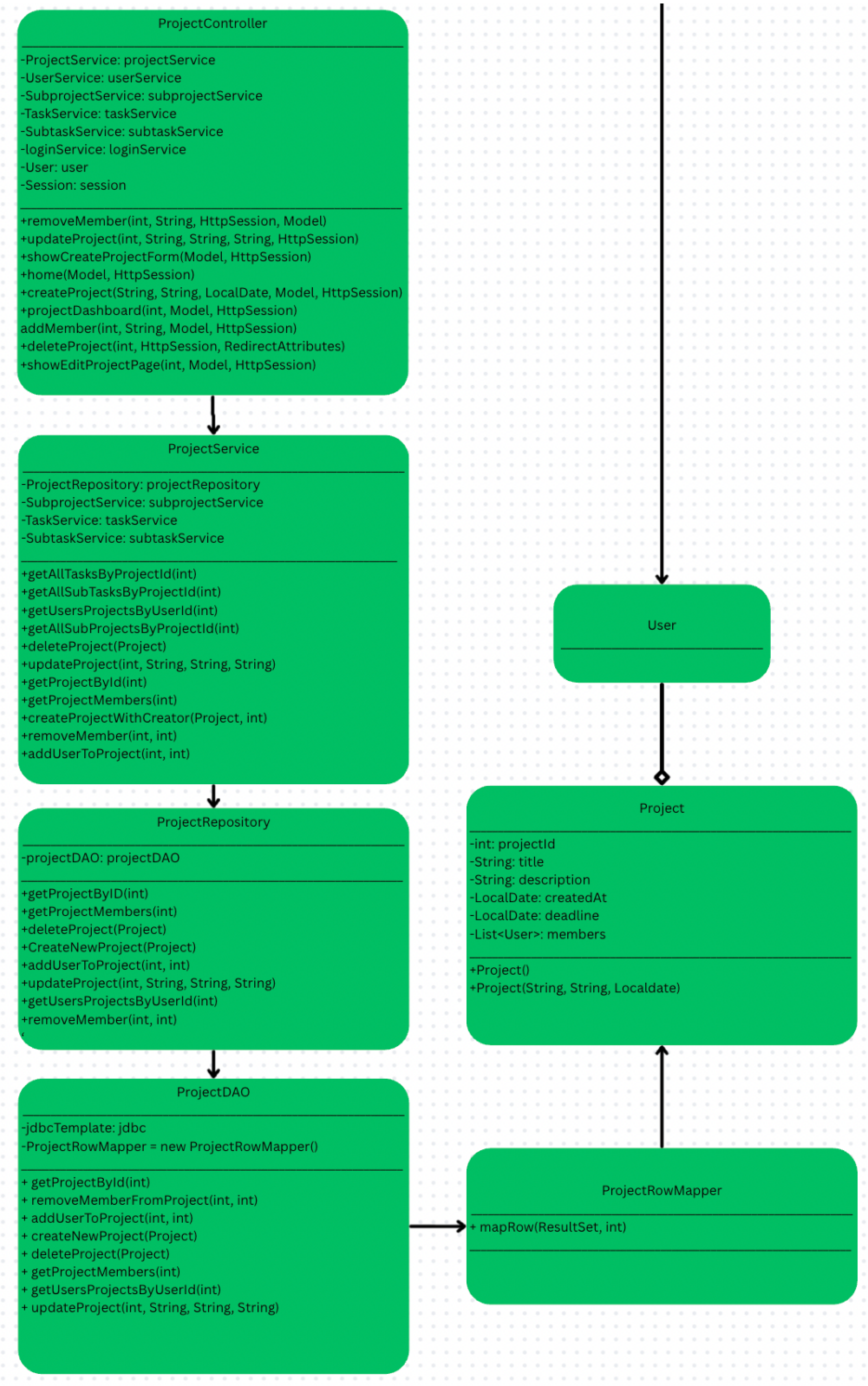
Bilag 5



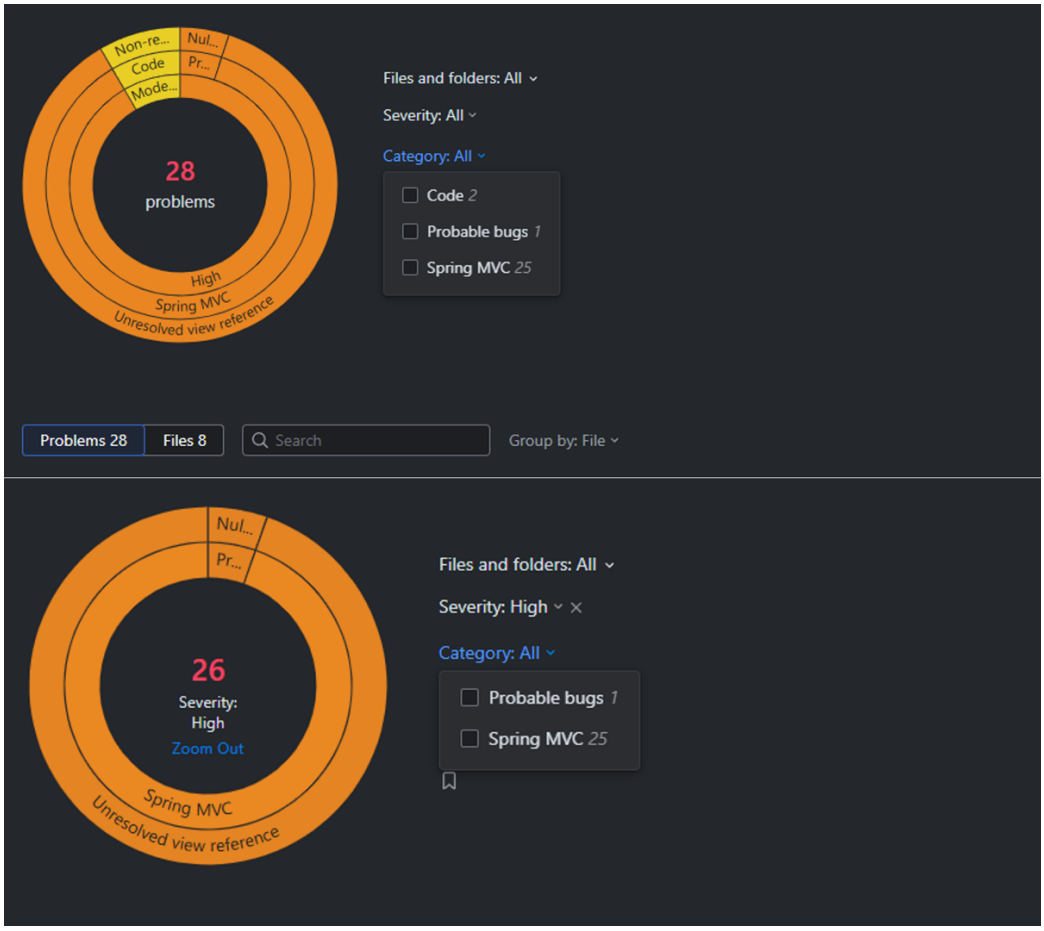
Bilag 6



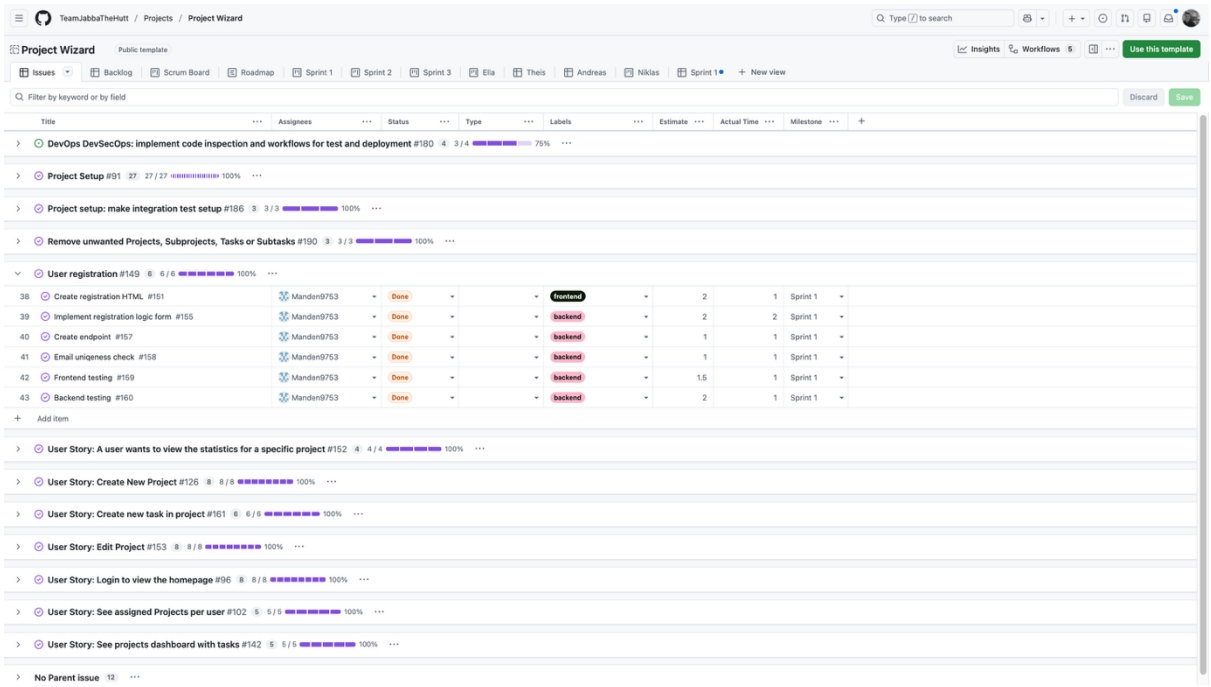
Bilag 7


ProjectController
-ProjectService: projectService
-UserService: userService
-SubprojectService: subprojectService
-TaskService: taskService
-SubtaskService: subtaskService
-loginService: loginService
-User: user
-Session: session
+removeMember(int, String, HttpSession, Model)
+updateProject(int, String, String, String, HttpSession)
+showCreateProjectForm(Model, HttpSession)
+home(Model, HttpSession)
+createProject(String, String, LocalDate, Model, HttpSession)
+projectDashboard(int, Model, HttpSession)
addMember(int, String, Model, HttpSession)
+deleteProject(int, HttpSession, RedirectAttributes)
+showEditProjectPage(int, Model, HttpSession)
ProjectService
-ProjectRepository: projectRepository
-SubprojectService: subprojectService
-TaskService: taskService
-SubtaskService: subtaskService
+getAllTasksByProjectId(int)
+getAllSubTasksByProjectId(int)
+getUsersProjectsByUserId(int)
+getAllSubProjectsByProjectId(int)
+deleteProject(Project)
+updateProject(int, String, String, String)
+getProjectById(int)
+getProjectMembers(int)
+createProjectWithCreator(Project, int)
+removeMember(int, int)
+addUserToProject(int, int)
User
ProjectRepository
-projectDAO: projectDAO
+getProjectByID(int)
+getProjectMembers(int)
+deleteProject(Project)
+CreateNewProject(Project)
+addUserToProject(int, int)
+updateProject(int, String, String, String)
+getUsersProjectsByUserId(int)
+removeMember(int, int)
Project
-int: projectId
-String: title
-String: description
-LocalDate: createdAt
-LocalDate: deadline
-List<User>: members
+Project()
+Project(String, String, Localdate)
ProjectDAO
-jdbcTemplate: jdbc
-ProjectRowMapper = new ProjectRowMapper()
+ getProjectById(int)
+ removeMemberFromProject(int, int)
+ addUserToProject(int, int)
+ createNewProject(Project)
+ deleteProject(Project)
+ getProjectMembers(int)
+ getUsersProjectsByUserId(int)
+ updateProject(int, String, String, String)
ProjectRowMapper
+ mapRow(ResultSet, int)

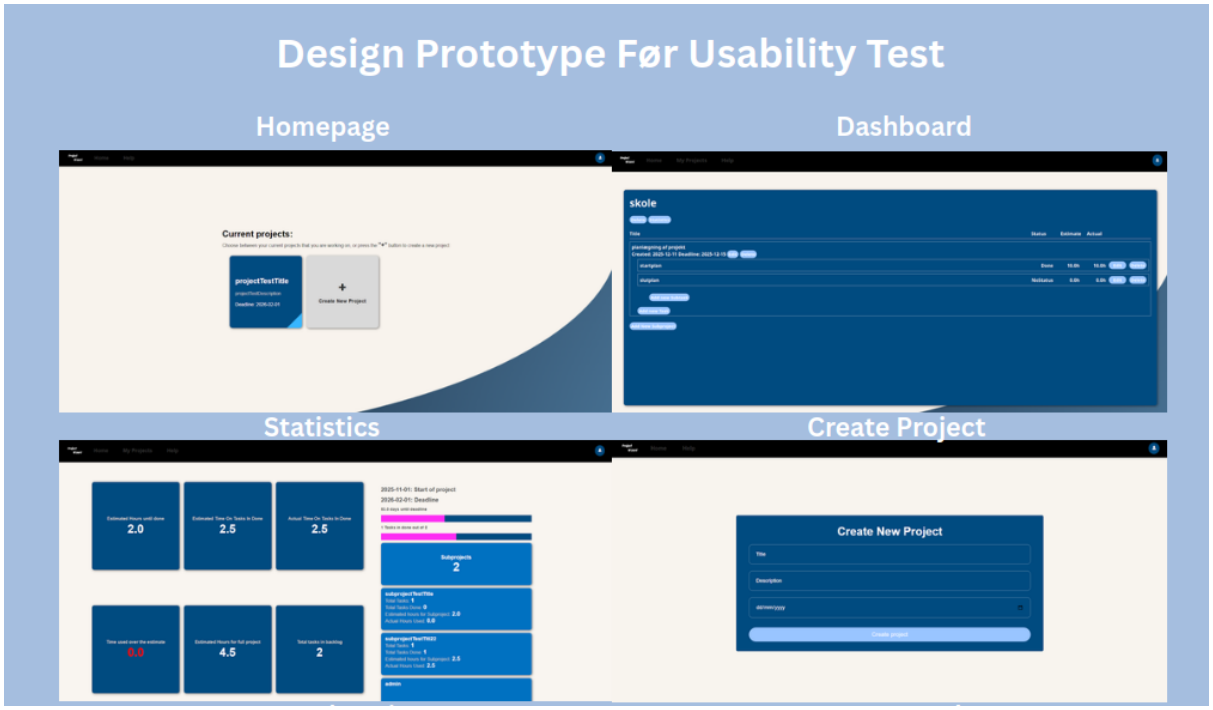
Bilag 8



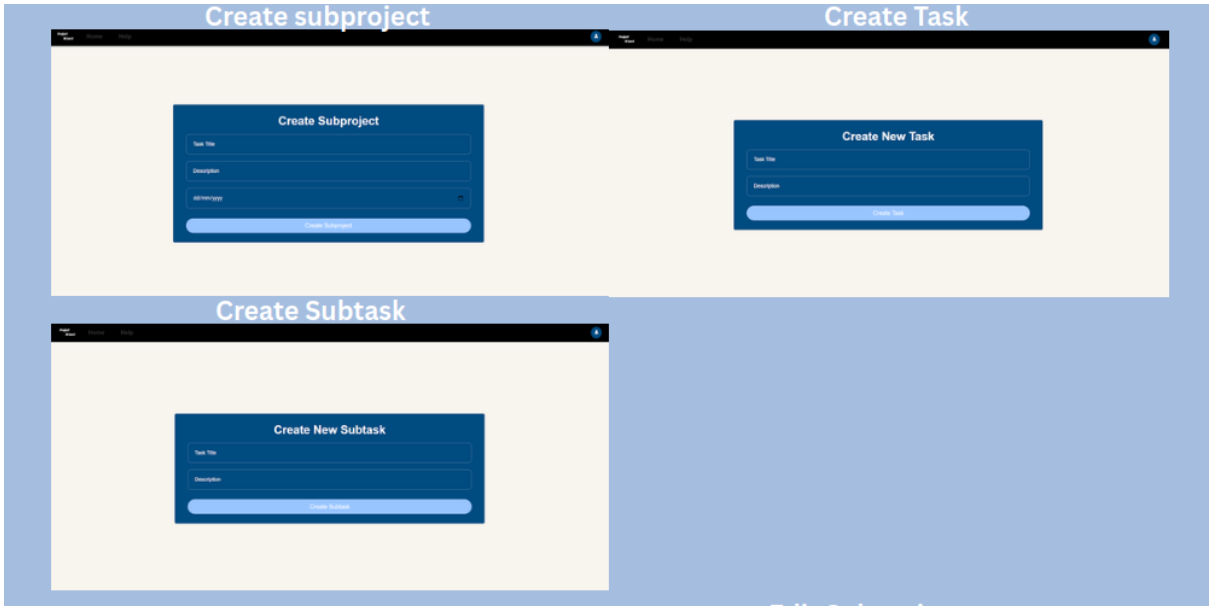
Bilag 9

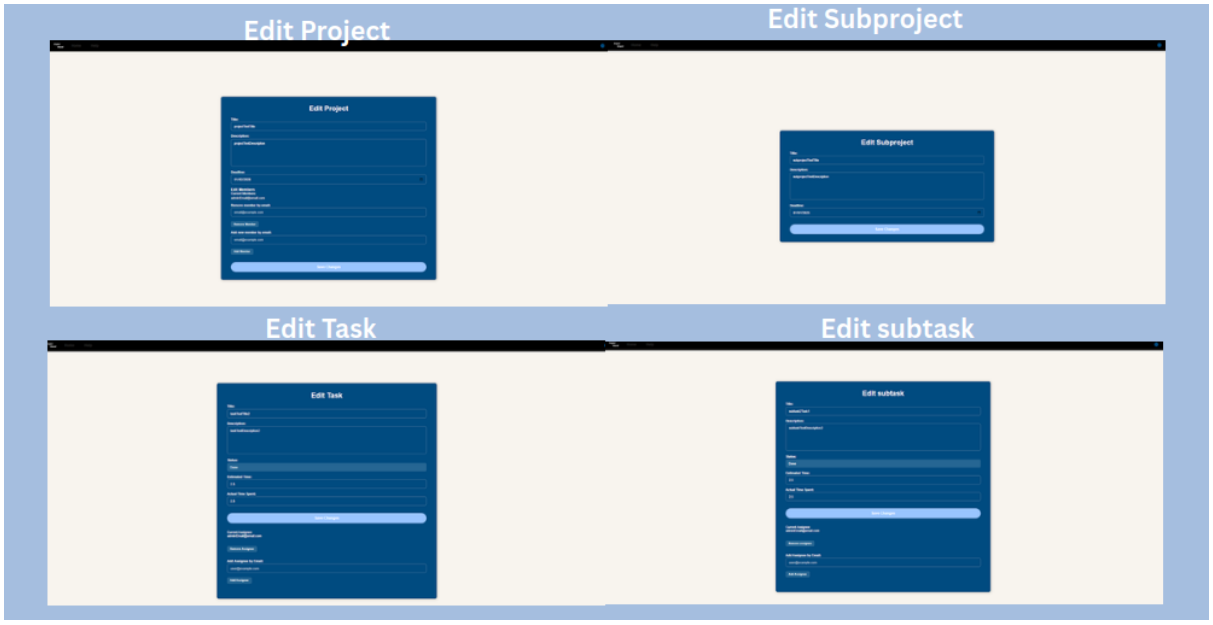


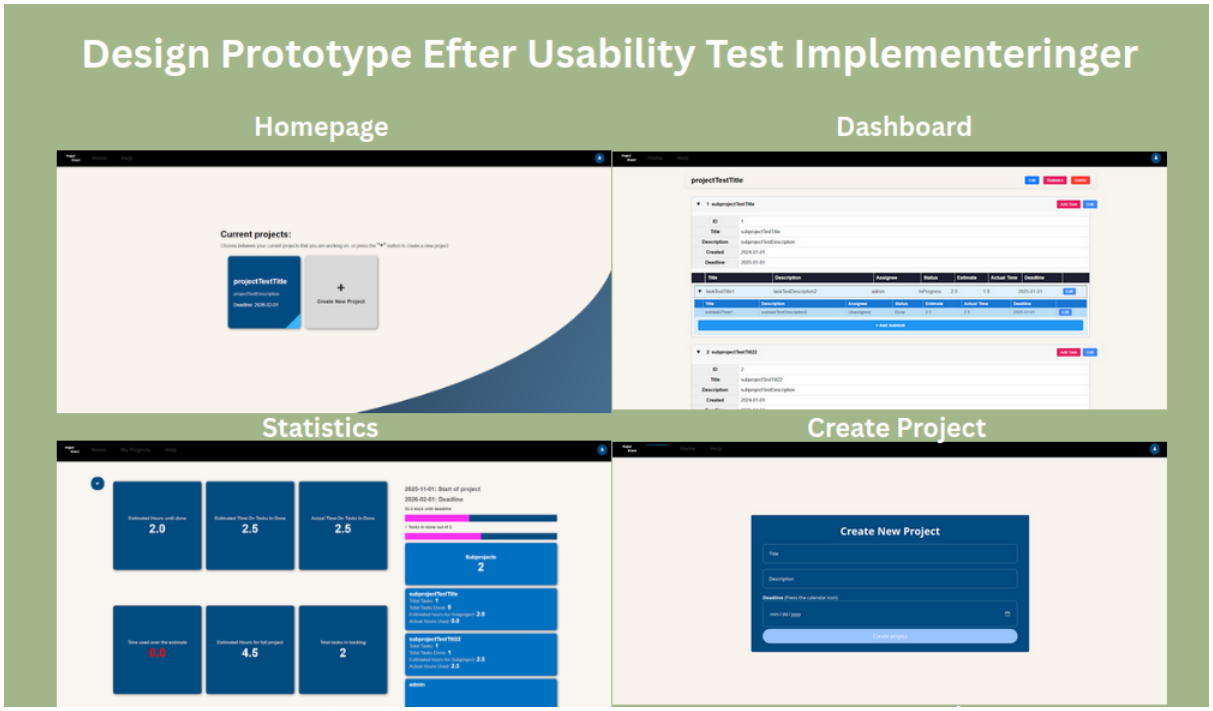
Bilag 10


Design Prototype Før Usability Test
Homepage
Current projects:
projectTestTitle
Create New Project
Dashboard
skole
Statistics
2.0
2.5
2.5
0.0
4.5
2
Create Project
Create New Project

Bilag 11

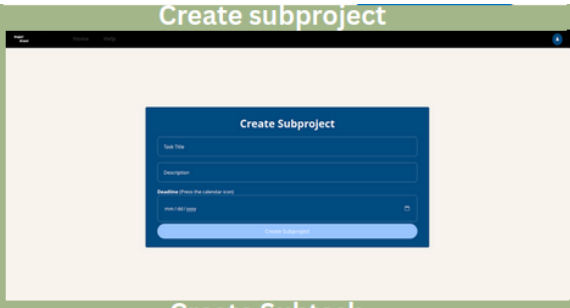

Create subproject
Create Subproject
Create Task
Create New Task
Create Subtask
Create New Subtask


Edit Project
Edit Project
Edit Subproject
Edit Subproject
Edit Task
Edit Task
Edit subtask
Edit subtask

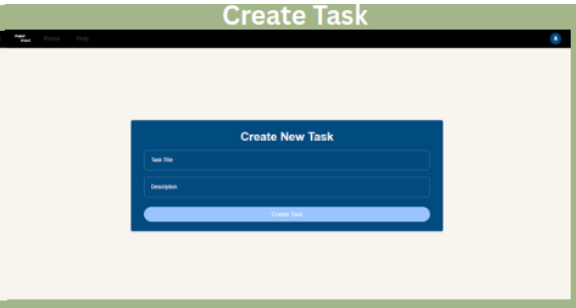

Design Prototype Efter Usability Test Implementeringer
Homepage
Current projects:
projectTestTitle
Create New Project
Dashboard
projectTestTitle
Statistics
2.0
2.5
2.5
2
0.0
4.5
2
Create Project
Create New Project

Bilag 12

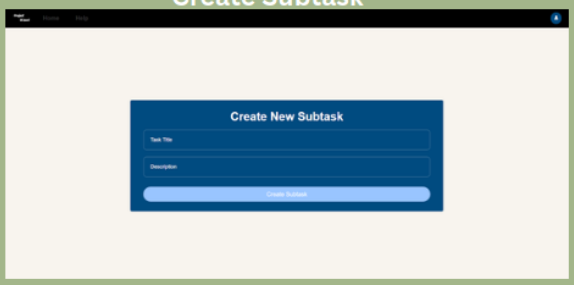
Create subproject



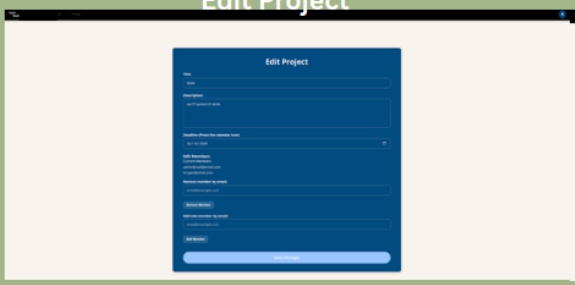
Create Task



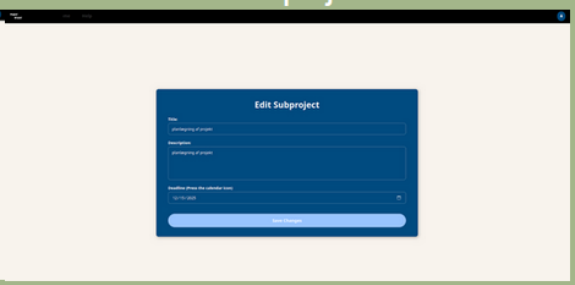
Create Subtask



Edit Project



Edit Subproject

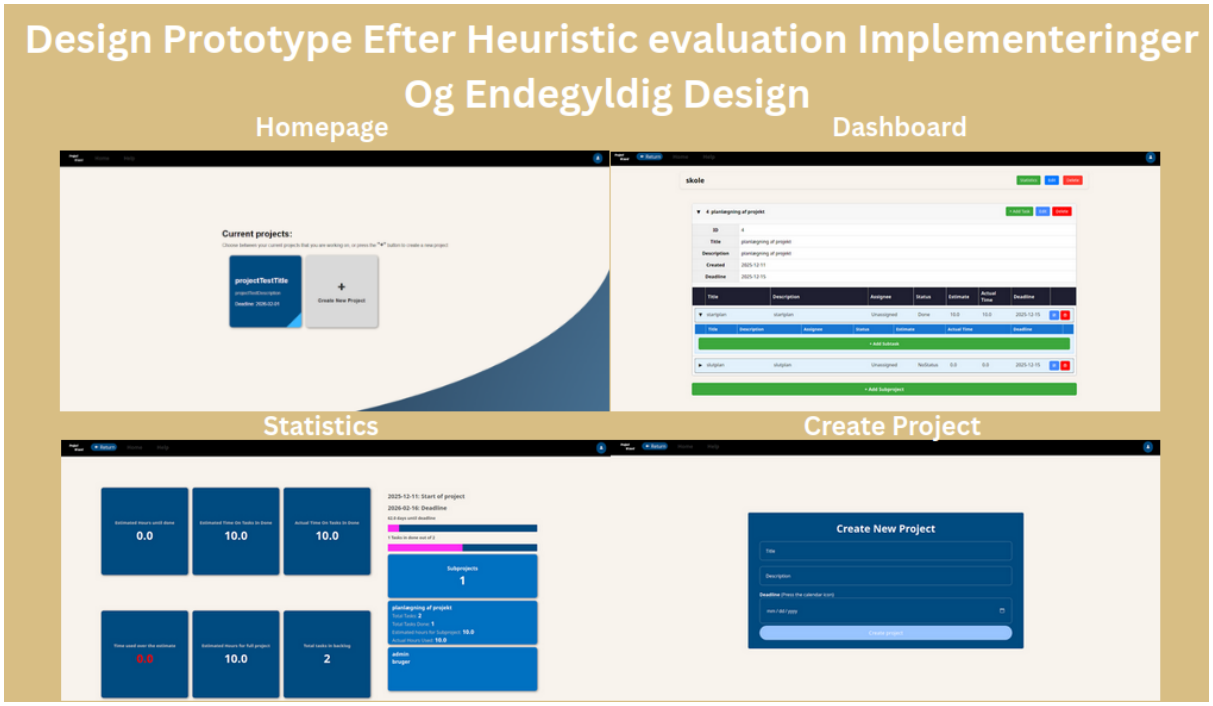


Edit Task

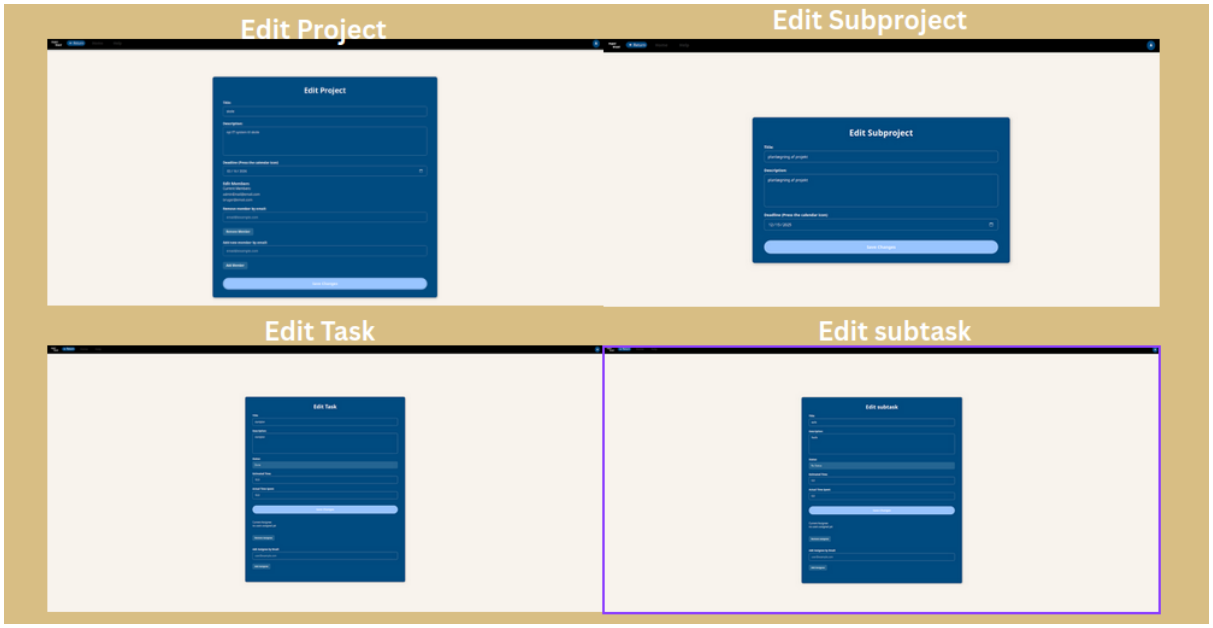


Edit subtask





Bilag 13



Heuristic	Issues	Recommendations
Visibility of System Status The design should always keep users informed about what is going on, through appropriate feedback	1. Der er ikke noget der indikerer hvilken stadie programmet er i. Såsom en overskrift der	Eventuelt indsætte en "loading" icon såfremt at systemet er ved at hente data til display

within a reasonable amount of time. • Does the design clearly communicate its state? • Is feedback present quickly after user actions? **Offer informative feedback**. For every operator action, there should be some system feedback. For frequent and minor actions, the response can be modest, while for infrequent and major actions, the response should be more substantial.	beskriver hvilken side brugeren netop er på. 2. Der er ikke nogen "Loading state" der viser at programmet er ved at hente informationer	Overskrifter i navbaren eller over vinduets content der beskriver hvilken side brugeren befinder sig på.
Match between system and the real world The design should speak the users' language. Use words, phrases, and concepts familiar to the user, rather than internal jargon. Follow real-world conventions, making information appear in **a natural and logical order.** • Will user be familiar with the terminology used in the design? • Do the design's controls follow real-world conventions?	3. Edit, Statistics og Delete knappernes farver er forvirrende. Delete kan forblive rød, men statistics skal ikke være lyserød, da det stadig kan være en skræmmende farve.	Ændre farven på statistics til grøn og edit til blå, lad delete forblive rød så den tydelig indikerer en definitiv handling.
User Control and Freedom Users often perform actions by mistake. They need a clearly marked "emergency exit" to leave the unwanted action without having to go through an extended process • Does the design allow users to go back a step in the process?	4. Brugeren skal navigere tilbage ved hjælp af sidens tilbage knapper, hvilket kan skabe forvirring. 5. Ingen tydelig "Cancel" knap på create og edit sider	Implementer tilbage knap på siderne, der tager brugeren ét step tilbage i deres process. Implementer en tydelig "Cancel" knap på edit og create siderne.

• Are exit links easily discoverable? • Can users easily cancel an action? • Is Undo and Redo supported? **Permit easy reversal of actions** This feature relieves anxiety, since the user knows that errors can be undone; it thus encourages exploration of unfamiliar options. The units of reversibility may be a single action, a data entry, or a complete group of actions.		
Consistency and Standards Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform and industry conventions. • Does the design follow industry conventions? • Are visual treatments used consistently throughout the design? **Strive for consistency** Consistent sequences of actions should be required in similar situations; identical terminology should be used in prompts, menus, and help screens; and consistent commands should be employed throughout	6. Alle projekter har samme farve	Eventuelt implementer at man selv kan vælge hvilken farve et projekt skal have på home page, så man eventuelt nemmere ville kunne finde frem til dem.
Error Prevention Good error messages are important, but the best designs carefully prevent problems from occurring in the first place. Either	7. Ikke alle Error former er dækket af custom exceptions	flere Error handling former kunne blive implementeret for at sikre fuld kontrol over fejl.

eliminate error-prone conditions, or check for them and present users with a confirmation option before they commit to the action. • Does the design prevent slips by using helpful constraints? • Does the design warn users before they perform risky actions?	8. Delete har ikke en forsikrings stadie, kan resultere i at brugeren sletter et projekt eller andet ved en fejl	Implementer en “are you sure” knap inden programmet sletter elementer.
Recognition Rather Than Recall Minimize the user's memory load by making elements, actions, and options visible. The user should not have to remember information from one part of the interface to another. Information required to use the design (e.g. field labels or menu items) should be visible or easily retrievable when needed. • Does the design keep important information visible, so that users do not have to memorize it? • Does the design offer help in-context? **Reduce short-term memory load** The limitation of human information processing in short-term memory requires that displays be kept simple, multiple page displays be consolidated, window-motion frequency be reduced, and sufficient training time be allotted for codes, mnemonics, and sequences of actions.	9. Ingen ‘help in context’ er udført udover på homepage. Dashboard og statistics mangler flere oplysninger.	Implementer tekst eller mere udførlige overskriver på dashboard og statistics der angiver præcist hvad det er brugeren ser.
Flexibility and Eciency of Use Shortcuts — hidden from novice users — may speed up the		Eventuelt implementer “last worked on” eller en liste over de projekter

interaction for the expert user such that the design can cater to both inexperienced and experienced users. Allow users to tailor frequent actions • Does the design provide accelerators like keyboard shortcuts and touch gestures? • Is content and funtionality personalized or customized for individual users? **Enable frequent users to use shortcuts** As the frequency of use increases, so do the user's desires to reduce the number of interactions and to increase the pace of interaction. Abbreviations, function keys, hidden commands, and macro facilities are very helpful to an expert user.	10. Ingen shortcuts til gengangere af programmet.	som brugeren har brugt tidligere eller sidst, der gør at de hurtigt kan navigere hen til det projekt de sidst havde arbejdet på uden at skulle søge i homepage.
Aesthetic and Minimalist Design Interfaces should not contain information that is irrelevant or rarely needed. Every extra unit of information in an interface competes with the relevant units of information and diminishes their relative visibility • Is the visual design and content focused on the essentials? • Have all distracting, unnescessary elements been removed?		
Help Users Recognize, Diagnose, and Recover from Errors Error messages should be expressed in plain language (no error codes), precisely indicate the	11. ikke håndterede error codes	Implementer pages udover error 500 så brugeren let kan komme tilbage til et velkendt sted

problem, and constructively suggest a solution. • Does the design use traditional error message visuals, like bold, red text? • Does the design offer a solution that solves the error immediately? **Offer simple error handling** as much as possible, design the system so the user cannot make a serious error. If an error is made, the system should be able to detect the error and offer simple, comprehensible mechanisms for handling the error.		
Help and Documentation It's best if the system doesn't need any additional explanation. However, it may be necessary to provide documentation to help users understand how to complete their tasks. • Is help documentation easy to search? • Is help provided in context right at the moment when the user requires it?	12. Help siden er for overfladisk	Uddyb eventuelle processer, så som hvordan tilføjelsen af andre brugere til et projekt påvirker deres homepage.

Design dialog yo yield closure Sequences of actions should be organized into groups with a beginning, middle, and end. The informative feedback at the completion of a group of actions gives the operators the satisfaction of accomplishment, a sense of relief, the signal to drop contingency plans and options from their minds, and an indication that the way is clear to prepare for the next group of actions		
Support internal locus of control Experienced operators strongly desire the sense that they are in charge of the system and that the system responds to their actions. Design the system to make users the initiators of actions rather than the responders		

Bilag 14